

[기업요구 반영]
지능형 웹서비스 개발(with JAVA, Spring)

UNICOOK

[개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰]

A-TEAM

박윤희, 박세연, 박정희, 백세원, 정정원, 진선용

2026.04.

목 차

- 01 프로젝트 개요
- 02 프로젝트 팀 구성 및 역할
- 03 프로젝트 수행 절차 및 방법
 - 03-1 요구사항분석내역
 - 03-2 기능분석
 - 03-3 디렉토리 구성도
 - 03-4 데이터베이스설계
 - 03-5 유스케이스 다이어그램
 - 03-6 시퀀스 다이어그램
 - 03-7 서비스 구성도
- 04 프로젝트 수행 경과
- 05 프로토타입 설계화면



01 UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰) 프로젝트 개요

01

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

프로젝트 개요

1

프로젝트 주제 및 선정 배경, 기획의도

1~2인 가구의 증가와 시생비의 시대의 전통적인 식재료 소비를 최소화하고 필요한 만큼만 제공되는 밀키트와 HMR(가정 간편식)의 새로운 기준을 제시

2

프로젝트 내용

간편식 쇼핑몰 선정, 일상생활에 흔히 접할 수 있으며 남녀노소에 관계없이 친숙한 주제, 고객 맞춤 추천 알고리즘을 적용하여 적절한 난이도와 파이썬 기반 작업으로 진행

3

활용 장비 및 재료

- **개발환경**: Windows 10 / Chrome / Git Hub / Spyder
- **개발도구**: Flask / exERD / StarUML / MySQL / PowerPoint / BootStrap
- **프로그래밍 언어**: python / JS / jQuery / AJAX / CSS / MySQL / Jinja2

4

프로젝트 구조

- **Main**: 메인 홈페이지_시간대별 맞춤 추천 상품
- **상세페이지**: 해당 상품과 연관된 상품 추천
- **장바구니**: 장바구니 연관 분석 추천 알고리즘
- **구매내역**: 기간 맞춤 추천

5

활용방안 및 기대 효과

소비자들의 요리하는 시간을 줄이고 식사를 즐기는 시간을 극대화하며 연관 상품 추천으로 추가 구매를 유도하고 소비자가 편의를 느낄 수 있도록 기대

02 UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰) 프로젝트 팀 구성 및 역할

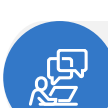
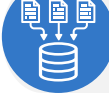
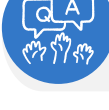


02

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

프로젝트 팀 구성 및 역할

- ▶ 해당 프로젝트를 진행하면서 모든 분야에 **훈련생 모두 적극적으로 협업하였습니다.**
그 중에서도 **주도적으로 참여한 부분을 중심으로** 역할을 작성하였습니다.

훈련생	역할	담당 업무	
박윤희	팀장	 프로젝트 총괄, prototype	 UI/UX 담당, css 작업
백세원	팀원	 데이터베이스 설계	 UI/UX 담당, 화면 기능구현
정정원	팀원	 프로젝트수행 계획서 작성	 화면 및 기능구현, PPT
진선용	팀원	 팀 회의록 작성, 데이터베이스 설계	 화면 및 기능 구현
박세연	팀원	 요구분석서 작성	 유스케이스, 시퀀스 작업
박정희	팀원	 prototype	 화면 및 기능구현, PPT
이현우	멘토	 프로젝트 질의응답	



03 UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰) 프로젝트 수행 절차 및 방법



03

K-Digital Training

프로젝트 수행 절차 및 방법

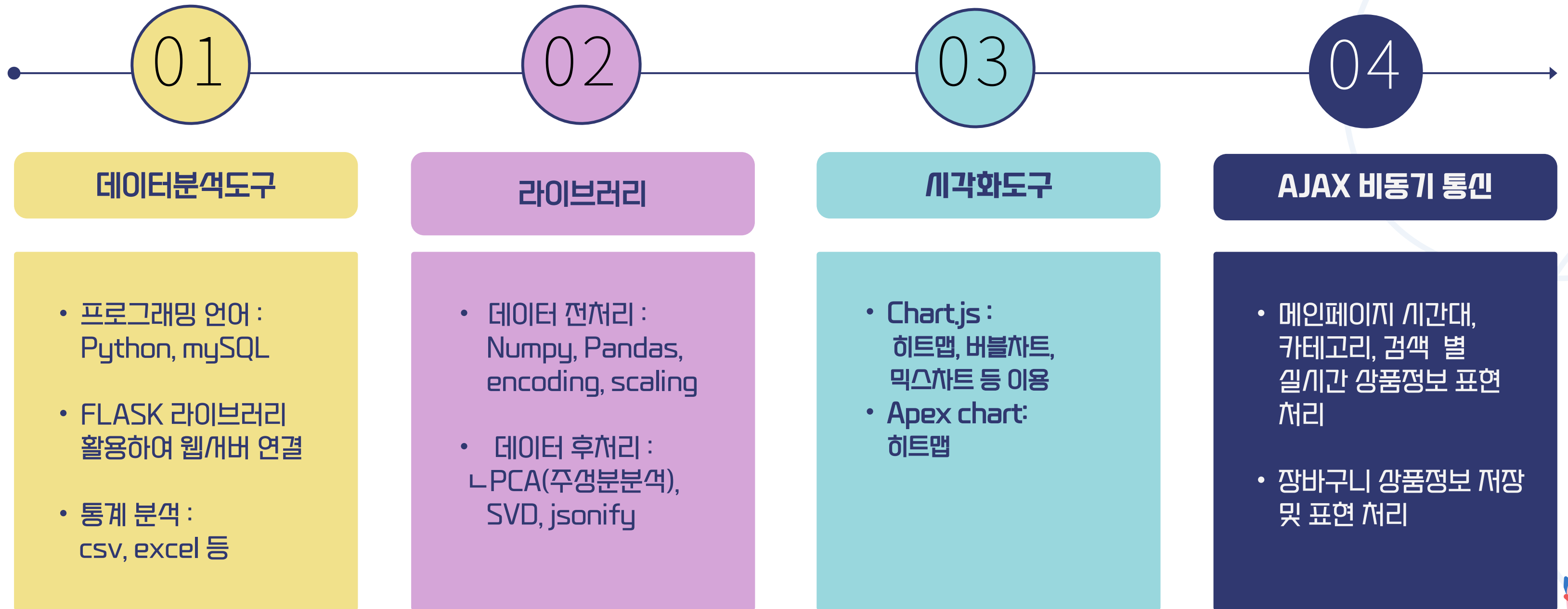
- ▶ 프로젝트의 **사전 기획**과 **프로젝트 수행 및 완료** 과정으로 나누어서 작성함.

구분	기간	활동	비고
사전 기획	3/24(화) ~ 3/26(목)	 프로젝트 기획 및 주제 선정  기획안 작성	아이디어 선정
데이터 수집 및 전처리	3/26(목) ~ 3/31(화)	 필요 데이터 및 수집 절차 정의  외부 데이터 수집  데이터 정제 및 정규화	관련사이트 데이터 참조
모델링	4/1(수) ~ 4/8(수)	 HTML 구현	
자료 작성 및 중간보고	4/9(목) ~ 4/10(금)	 요구분석서, 프로그램 계획서 작성	팀별 중간보고 실시
서비스 구축	4/13(월) ~ 4/24(금)	 데이터 분석 시각화 및 알고리즘	
총 개발기간	3/24(화) ~ 4/27(월)	총 5주 프로젝트 완성	

03-1

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

요구사항 분석내역 : 핵심기술 요구사항



x

03-1 UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

요구사항 분석내역 : 페이지별 기능 요구사항

페이지를 메인, 상세, 장바구니, 구매내역으로 분리하여 핵심 기능 정의

main

- 상단 상품 목록 조회 : 시간대별 맞춤 상품 추천
- 하단 상품 목록 조회 : 카테고리별 필터링, 최다 판매 상품 추천
- 검색 기능 : 검색한 키워드가 있는 상품 출력

상세

- 함께하면 더 맛있는 상품 추천 : 코사인 분석을 이용해 연관된 상품 알고리즘 사용
- 수량 증감 버튼 사용

장바구니

- 로그인 시 장바구니 화면 출력
- 장바구니에 담은 상품과 연관된 상품 추천
- 체크박스로 상품 선택
- 상품별 소계 및 총 결제금액 표시

구매내역

- 로그인 시 구매내역 화면 출력
- 기간 필터링 : 최근 1개월 / 3개월 / 전체
- 가장 많이 구매한 상품별로 분석 리포트 화면 출력

03-2

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

기능분석

3.1 회원 관리

3.1.1 로그인

- 회원 ID / 비밀번호 기반 로그인 처리 (POST /loginok.do)
- 로그인 성공 시 Flask Session에 사용자 정보(id, name) 저장
- 로그인 실패 시 'ERROR' 반환, 성공 시 'OK' 반환 (AJAX 처리)
- 로그인 모달(Modal) 형태로 UI 구현 (login.html)
- 드롭다운 선택 방식으로 테스트 계정(user_001 ~ user_005) 제공

3.1.2 로그아웃

- GET /logout.do 요청 시 세션 초기화 후 메인 페이지로 리다이렉트

3.1.3 로그인 상태 확인

- GET /loginCheck.do 호출 시 세션 존재 여부를 JSON으로 반환
- 응답 예: {"login": true} 또는 {"login": false}

3.2 상품 목록 조회 (메인 페이지)

- GET /: 메인 페이지에서 전체 상품 목록 출력
- 카테고리
필터(전체/샐러드·샌드위치·떡볶이·튀김·순대/밀키트/도시락·밥류/치킨·피자·핫도그·만두) 지원
- 페이지당 16개 상품 출력 (페이지네이션 5개 블록 단위)
- 시간대별 추천 상품 섹션 별도 표시 (상단 4개 고정)
- 카테고리 변경 시 AJAX로 _category_partial.html 부분 렌더링

3.3 카테고리 필터 및 검색

3.3.1 카테고리 필터

- GET /category.do?category={값} 요청 처리
- 카테고리 코드 0 = 전체, 1~5 = 각 카테고리
- 결과를 _category_partial.html 파셜 템플릿으로 반환

3.3.2 키워드 검색

- GET /search.do?keyword={검색어} 요청 처리
- 상품명 기준 키워드 검색, 결과를 파셜 템플릿으로 반환
- 검색 시 카테고리는 '전체(0)'로 고정

3.4 상품 상세 조회

- GET /view.do?code={상품코드} 요청 처리
- 상품코드(code) 미전달 시 메인 페이지로 리다이렉트
- 상품명, 이미지, 카테고리, 가격, 중량/판매단위 표시
- 수량 증감 버튼(QtyPlus / QtyMinus) 제공
- '장바구니 담기' 버튼 클릭 시 AJAX로 장바구니 추가
- '함께하면 더 맛있는 추천 음식' 섹션 표시 (연관 상품 4개 하드코딩)
- SVD 알고리즘 기반 연관분석 리포트 Modal 제공



3.5 장바구니

3.5.1 장바구니 조회

- GET /cart.do : 로그인 사용자의 장바구니 목록 표시
- 비로그인 시 빈 목록 표시 (로그인 유도)
- 각 상품별 이미지, 상품명, 가격, 수량 표시
- 전체 선택 체크박스 및 개별 체크박스 제공
- 우측 결제금액 패널: 상품별 소계 및 총 결제금액 표시

3.5.2 장바구니 추가

- GET /cartadd.do?code={상품코드}&qty={수량} 요청 처리
- 비로그인 시 JSON {"result": "fail", "message": "로그인이 필요합니다."} 반환
- 추가 성공 시 최신 장바구니 건수(new_count) 함께 반환
- 헤더 장바구니 카운트 실시간 업데이트

3.5.3 장바구니 삭제

- POST /cartdelete.do : 선택한 항목(cno 배열) 일괄 삭제
- 삭제 후 최신 장바구니 건수 반환하여 헤더 카운트 갱신
- 개별 삭제 버튼(X) 및 선택삭제 버튼 모두 지원

3.6 구매 내역

3.6.1 구매 내역 조회

- GET /purchase.do : 로그인 사용자의 구매 내역 목록 표시
- 기간 필터: 최근 1개월 / 3개월 / 전체 (period 파라미터)
- 페이지당 5건 표시 (페이지네이션 5개 블록 단위)
- 각 주문: 구매일시, 주문번호, 상품이미지, 상품명, 단가, 총 결제금액, 수량 배지 표시
- 구매 내역이 없을 때 안내 메시지 및 쇼핑 버튼 표시

3.6.2 구매 완료 처리

- POST /purchase.do : 장바구니에서 선택 상품 구매 처리
- 구매 처리 후 해당 상품을 장바구니에서 자동 삭제
- 비로그인 시 JSON 오류 메시지 반환

3.7 식단 분석 리포트 (bunsuk)

- GET /bunsuk.do?target={대상} 요청 처리
- target 파라미터에 따라 다른 분석 Modal 제공: main / mainsub / view / cart
- main: 파이 차트(Chart.js)를 활용한 개인별 식단 분석 리포트
- view: 막대 차트를 활용한 SVD 알고리즘 연관분석 리포트
- cart: 장바구니 탄단지 균형 분석 결과
- 구매 페이지: 식단 균형 점수, 단백질 비중, 채소 비중, 영양 비율 프로그레스 바 표시

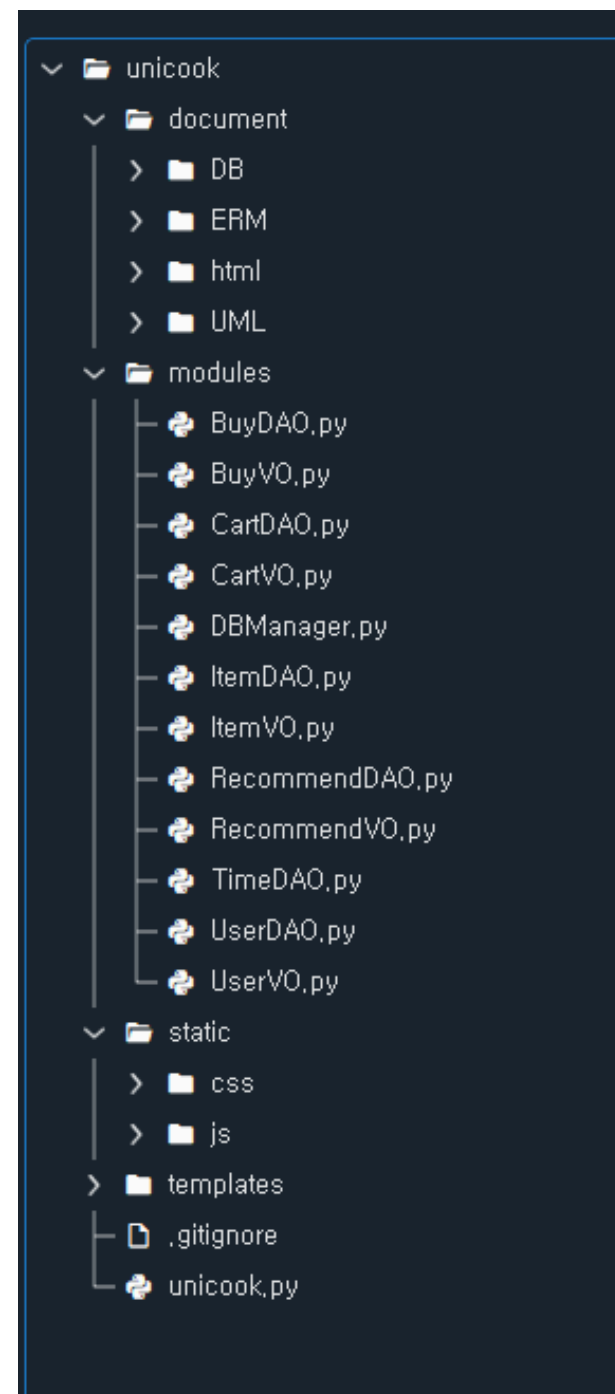
03-3

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

디렉토리 구성도

1. 프로그램 디렉토리

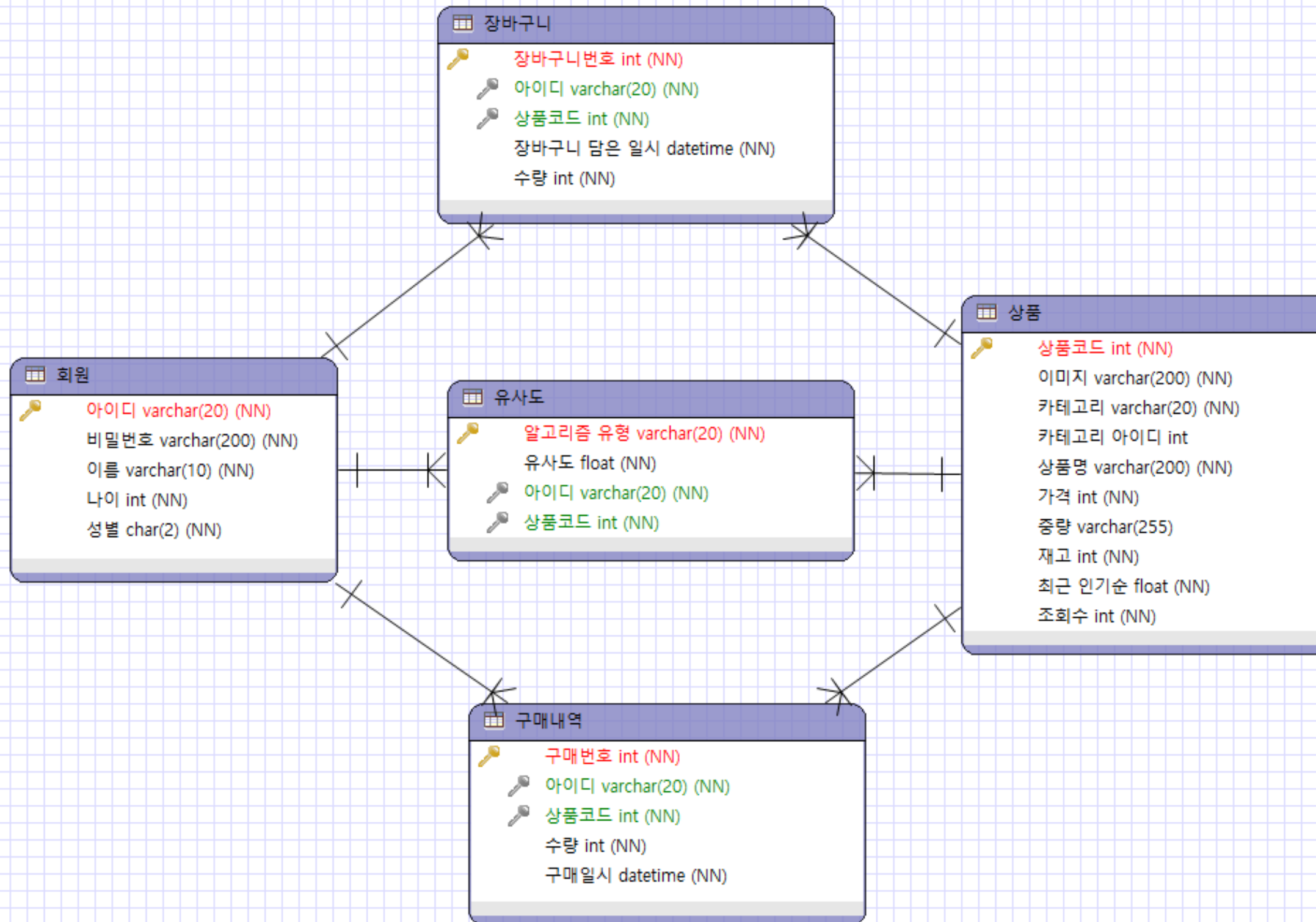
디렉토리명	설명	비고
/unicook/document/DB	데이터베이스 설계 문서 관리 폴더	
/unicook/document/ERM	논리 물리 erm 문서 관리 폴더	
/unicook/document/html	html 원본 파일	
/unicook/document/UML	unicook 시퀀스 문서	
/unicook/modules	DAO, VO 문서 관리	
/unicook/static/css	CSS 관리 폴더	
/unicook/static/js	jQuery, js 관리 폴더	
/unicook/templates	화면 출력 html 관리 폴더	
/unicook/templates/include	html include 통합 관리 폴더	
/unicook/unicook	unicook 메인 py 파일	



03-4

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

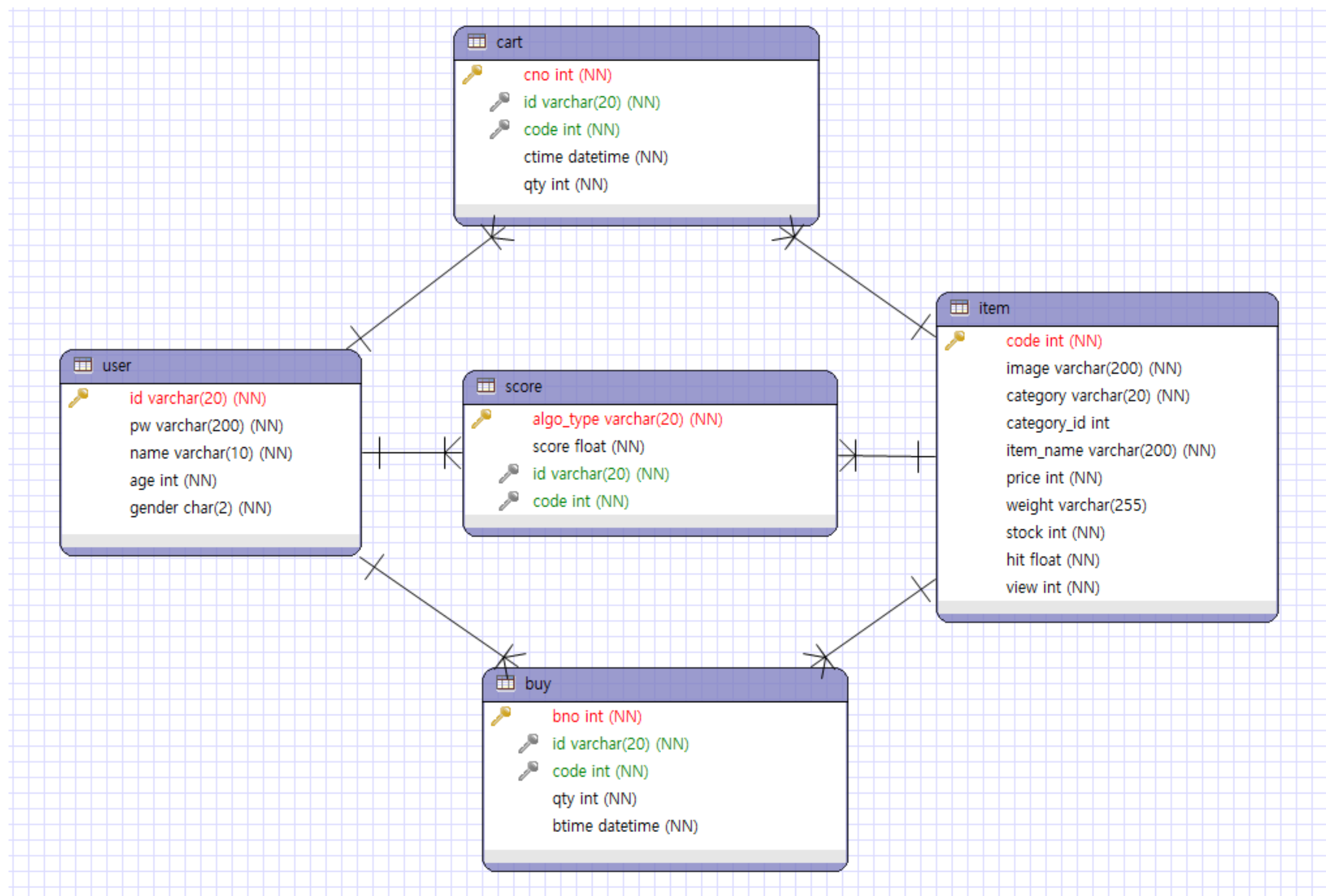
데이터베이스 설계 : 논리설계모델



03-4

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

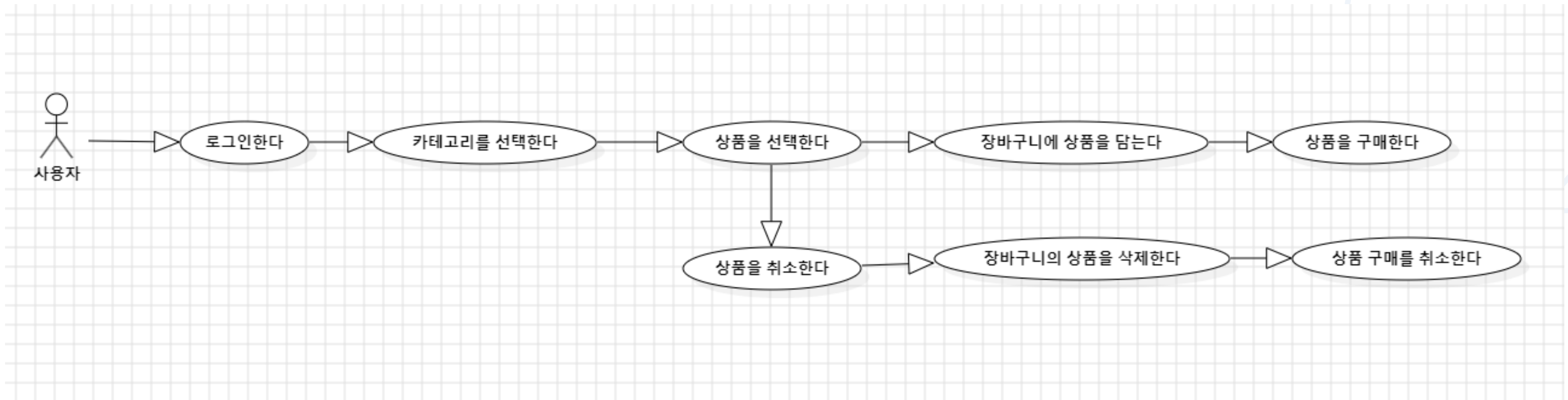
데이터베이스 설계 : 물리설계모델



03-5

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

유스케이스 다이어그램



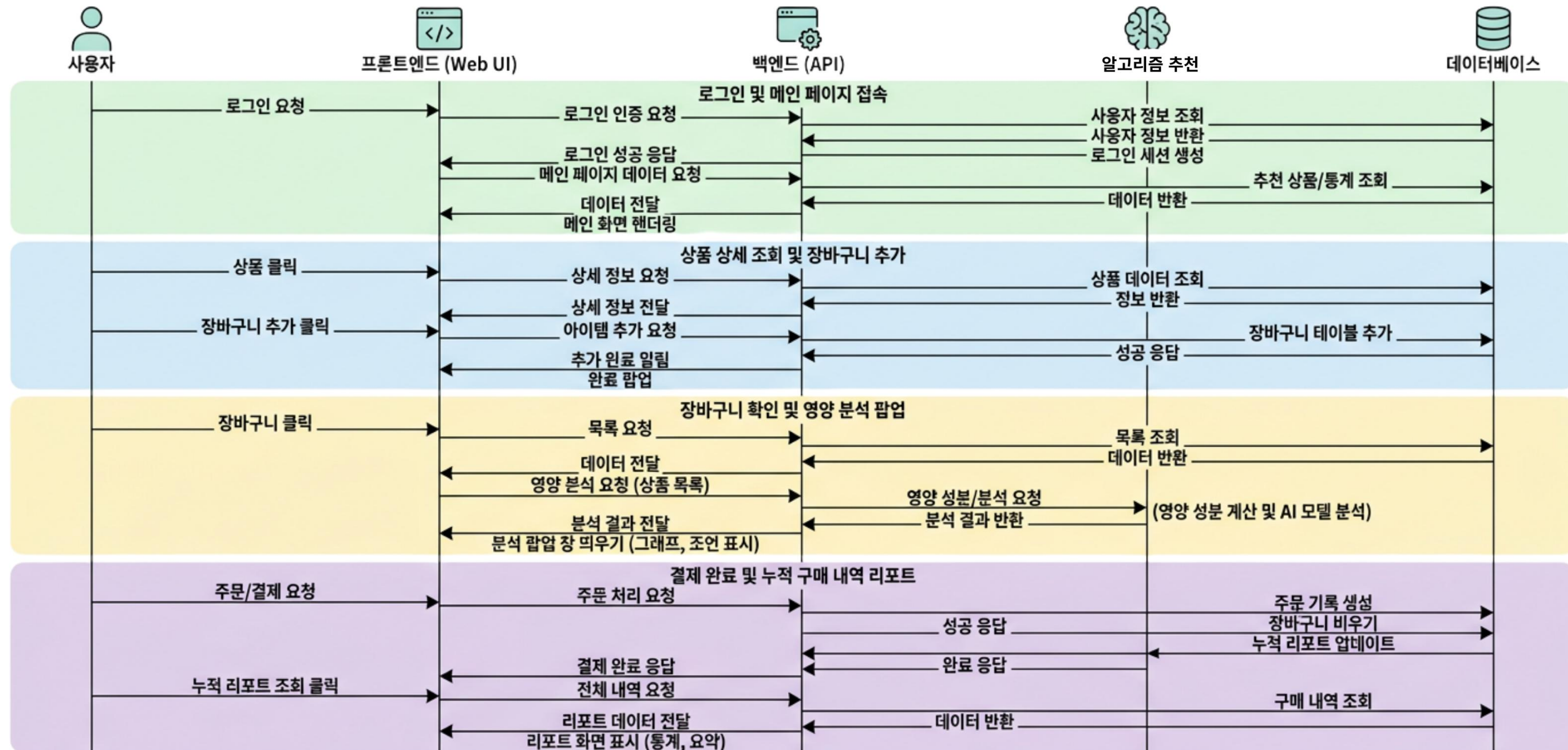
X

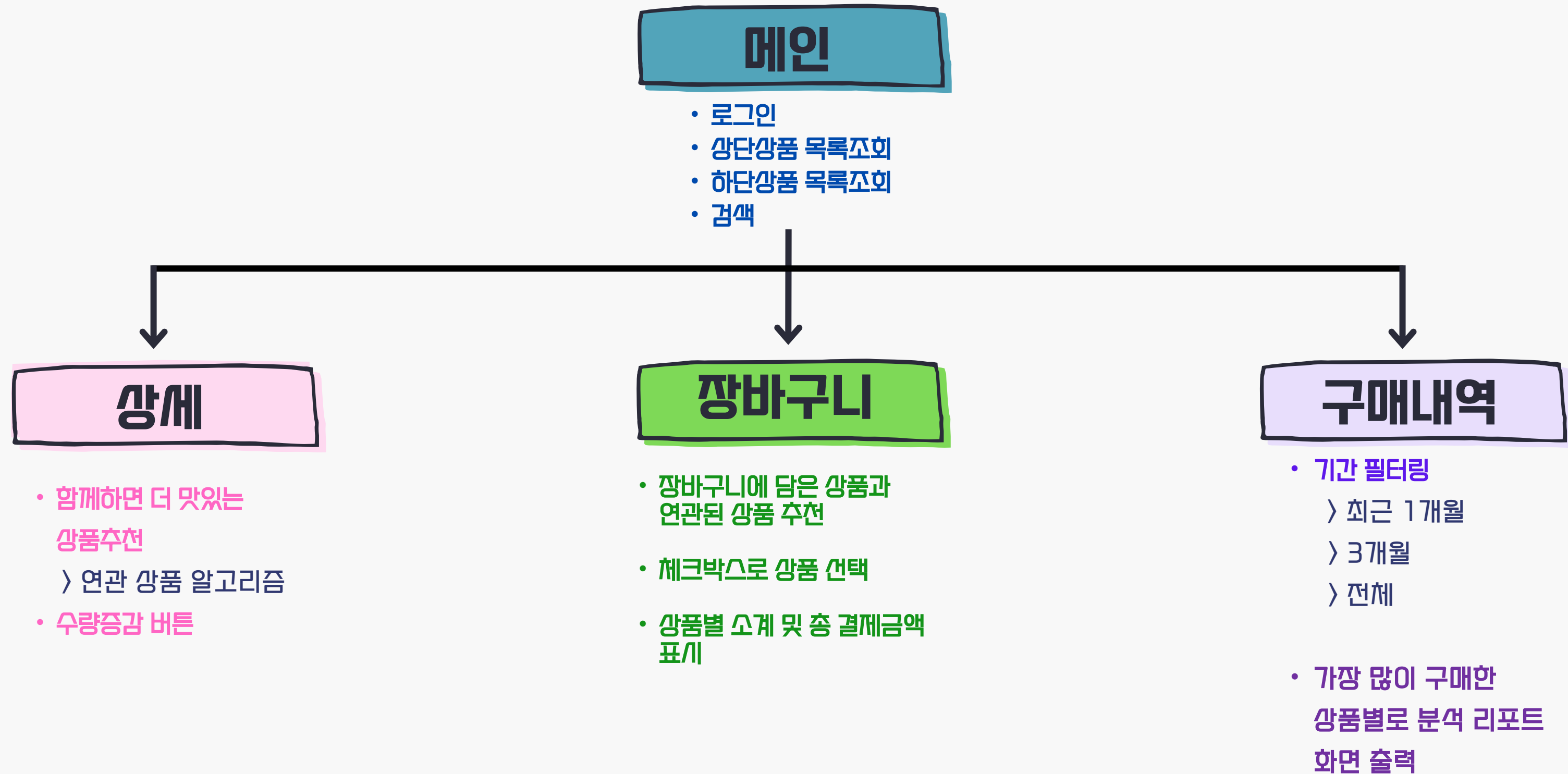
03-6

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

시퀀스 다이어그램 : 공통

식단 분석 서비스 시퀀스 다이어그램





04 UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰) 프로젝트 전체 일정



04-1 UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰) 프로젝트 일정표

▶ 개발기간 : 2026년 3월 24일(화) ~ 4월 27일(월)

WBS 번호	작업 제목	작업 소유자	시작일	마감일	기간	2026. 03						2026. 04																		
						24	25	26	27	30	31	1	2	3	6	7	8	9	10	13	14	15	16	17	20	21	22	23	24	27
						화	수	목	금	월	화	수	목	금	월	화	수	목	금	월	화	수	목	금	월	화	수	목	금	월
1	기획 및 요구사항분석																													
1.1	기획 및 기획 내용 작성	A팀	26년 3월 24일	26년 3월 25일	2																									
1.2	프로젝트 구현을 위한 스터디	A팀	26년 3월 25일	26년 3월 30일	4																									
1.3	요구사항 분석 및 문서화	A팀	26년 3월 27일	26년 3월 31일	3																									
2	수집																													
2.2	수집 시스템 설계	A팀	26년 3월 31일	26년 4월 2일	3																									
2.3	수집 시스템 구성	A팀	26년 3월 31일	26년 4월 2일	3																									
2.4	수집 및 변환 모듈 개발	A팀	26년 4월 3일	26년 4월 9일	5																									
2.5	수집 데이터 검증 모듈 개발	A팀	26년 4월 3일	26년 4월 9일	5																									
2.8	중간발표	A팀	26년 4월 10일	26년 4월 10일	1																									
3	저장																													
3.1	저장 모델 설계	A팀	26년 4월 13일	26년 4월 14일	2																									
3.2	저장 관리 시스템 구성	A팀	26년 4월 13일	26년 4월 14일	2																									
3.3	적재 모듈 개발	A팀	26년 4월 14일	26년 4월 17일	4																									
4	처리																													
4.1	처리시스템 설계	A팀	26년 4월 13일	26년 4월 14일	2																									
4.2	처리 시스템 구성	A팀	26년 4월 13일	26년 4월 14일	2																									
4.3	처리 시스템 수행 모듈 개발	A팀	26년 4월 14일	26년 4월 17일	4																									
5	분석																													
5.1	분석 시스템 설계	A팀	26년 4월 20일	26년 4월 21일	2																									
5.2	분석 시스템 구성	A팀	26년 4월 20일	26년 4월 21일	2																									
5.3	분석 모듈 개발 (분석 모듈 개발)	A팀	26년 4월 21일	26년 4월 24일	4																									
5.4	분석 모듈 개선	A팀	26년 4월 21일	26년 4월 24일	4																									
6	시각화																													
6.1	시각화 요건 정의와 스토리보드 기획	A팀	26년 4월 20일	26년 4월 21일	2																									
6.2	시각화 도구와 기술 선택	A팀	26년 4월 21일	26년 4월 24일	4																									
6.3	분석정보 시각화	A팀	26년 4월 21일	26년 4월 24일	4																									
7	완료																													
7.1	프로젝트 발표	A팀	26년 4월 27일	26년 4월 27일	1																									



프로젝트 수행과정

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

프로젝트 계획 및 준비

2026년 3월 24일
프로젝트의 전반적인 계획과 준비 작업수행

1

데이터베이스 및 HTML 화면구현 설계

2026년 4월 1-8일
DB 논리설계&물리설계 작업
와이어프레임을 토대로 HTML 웹페이지 구현

3

콘텐츠 개발 및 디자인

2026년 3월 26-31일
와이어프레임과 디자인, 기능구현 작업수행

2

중간보고 자료 작성 및 중간보고

2026년 4월 9-10일
PPT 발표 자료 작성 및 서류 검토

4

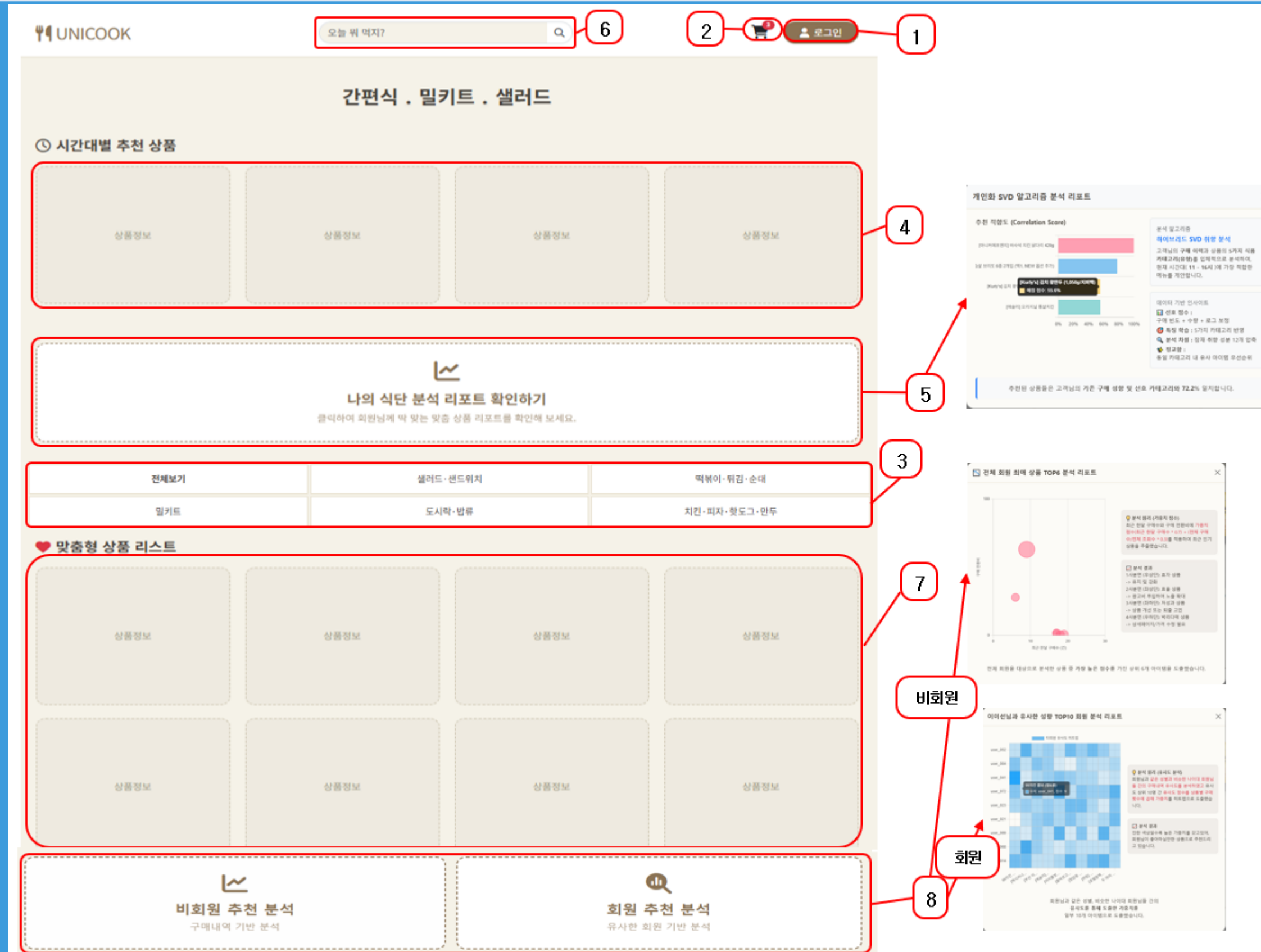
05 UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰) 프로토타입 설계화면



05-1

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

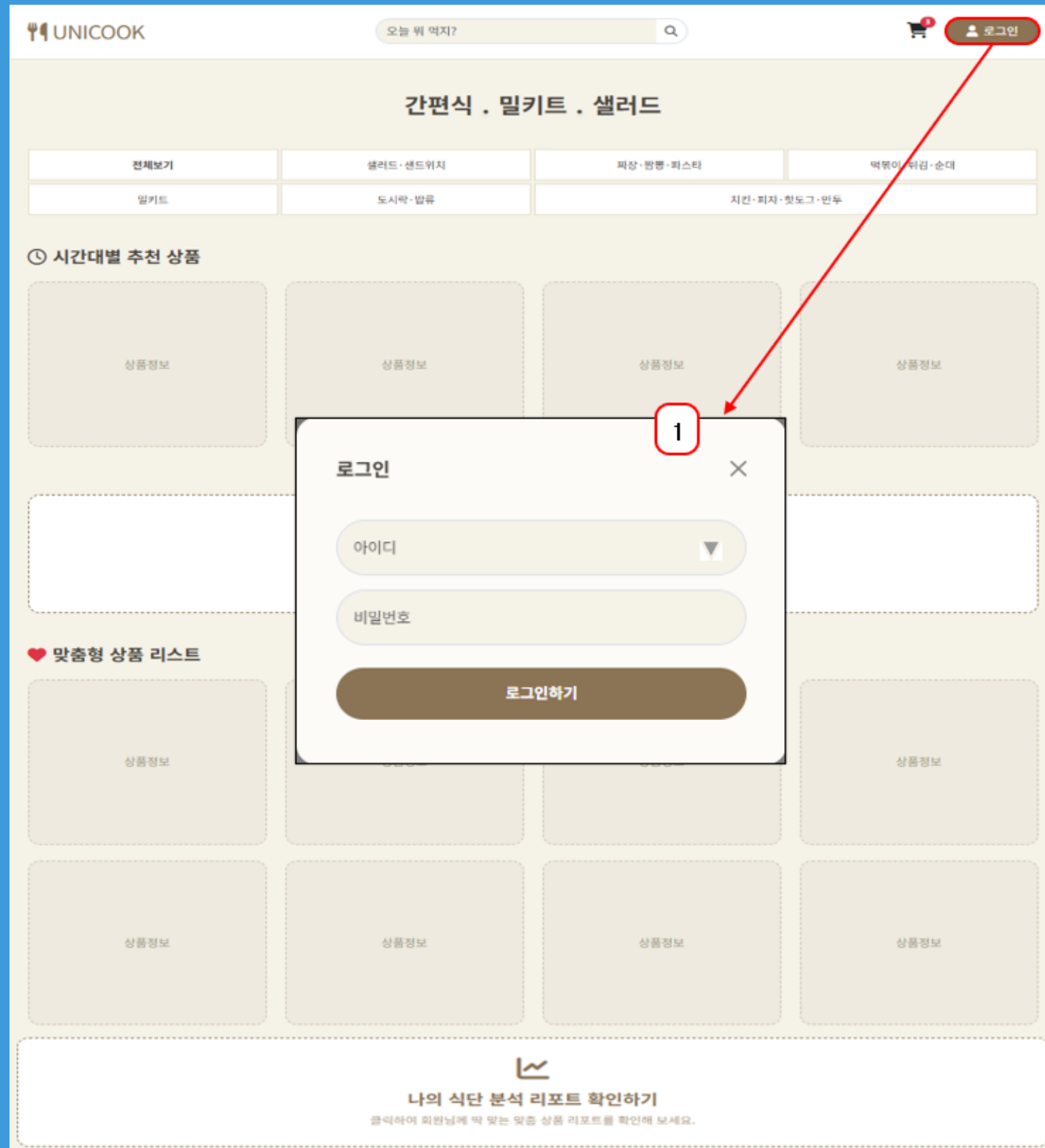
프로토타입 설계화면(메인페이지)



메인화면

- 1 -> 클릭 시 모달창 생성
* 임의 계정 생성하여 로그인
* 회원가입 따로 만들지 않음
- 2 -> 클릭 시 장바구니 페이지 이동
* 장바구니 버튼 로그인 시 활성화
- 3 -> 클릭 시 키워드에 맞는 상품 리스트 출력
- 4 -> 클릭 시 해당 상품의 상세페이지로 이동 : 시간대별 최다 구매
- 5 -> 클릭 시 코사인 유사도 알고리즘 이용한 콘텐츠 필터링으로 도출 과정 출력(시간대별 추천 상품)
- 6 -> 검색 키워드에 맞는 상품 리스트 현재 화면에 출력
- 7 -> 클릭 시 해당 상품의 상세 페이지로 이동
- 8 -> (비회원)클릭 시 최근 한달 구매 수와, 구매 전환비율 가중치를 구한 버블차트 (회원)연령과 성별로 유사도 구해 상품별 구매수 곱한 히트맵 과정 출력(맞춤형 상품 리스트)

05-2 UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰) 프로토타입 설계화면(로그인모달)



로그인 화면

- 1 -> 로그인 화면
DB에서 임의로 만들어둔
User Data를 가져와 표시하고
드롭 다운으로 출력함.

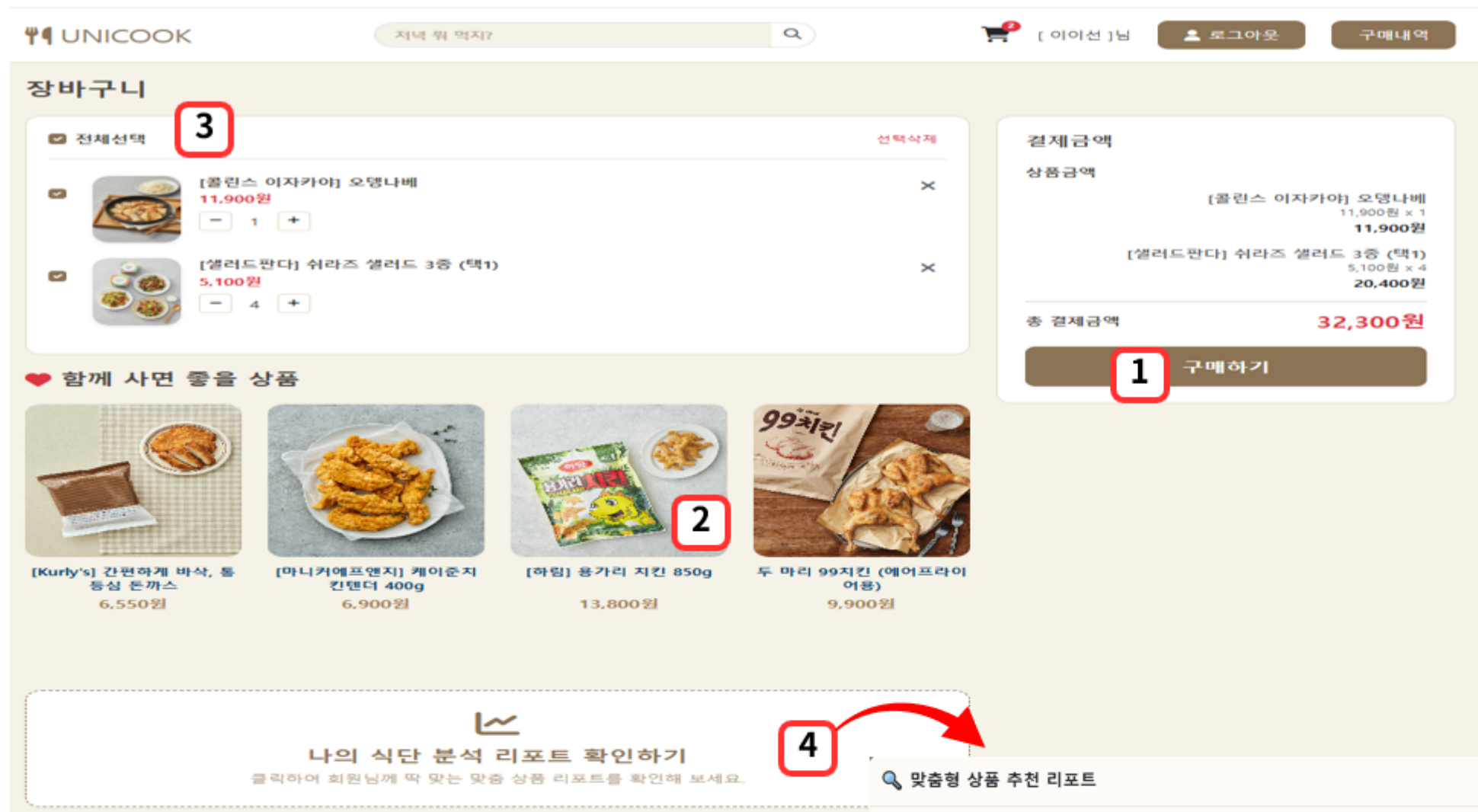
프로토타입 설계화면(상세페이지)

상세 화면

- 1 -> 클릭 상품 상세 페이지로 이동 : **관련상품(성별/연령 기준)**
- 2 -> 클릭 시 장바구니에 현재 선택한 상품 담김
- 3 -> 구매 내역으로 이동
- 4 -> 클릭 시 4번의 상품들을 도출한 과정 출력 : **SVD 알고리즘** 사용한 **협업 필터링**



05-4 UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰) 프로토타입 설계화면(장바구니페이지)

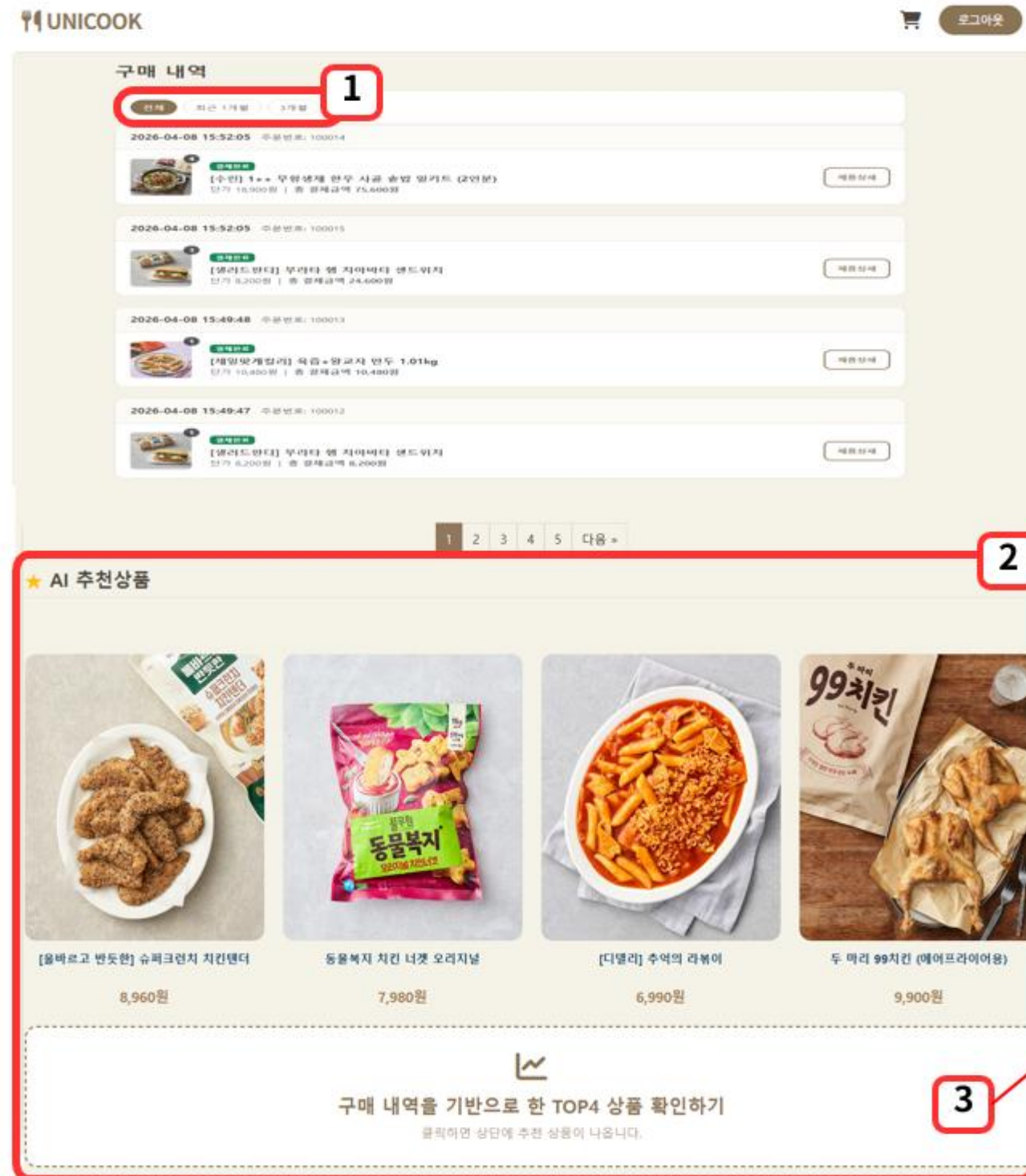


장바구니 화면

- 1 구매 결정을 내린 후 구매하기 버튼을 눌러 상품을 구입
- 2 비슷한 소비자들이 함께 구매한 상품 추천
: 관련상품(장바구니 상품기준)
- 3 장바구니에 담긴 상품
- 4 협업 필터링 분석 결과를 보여준다.

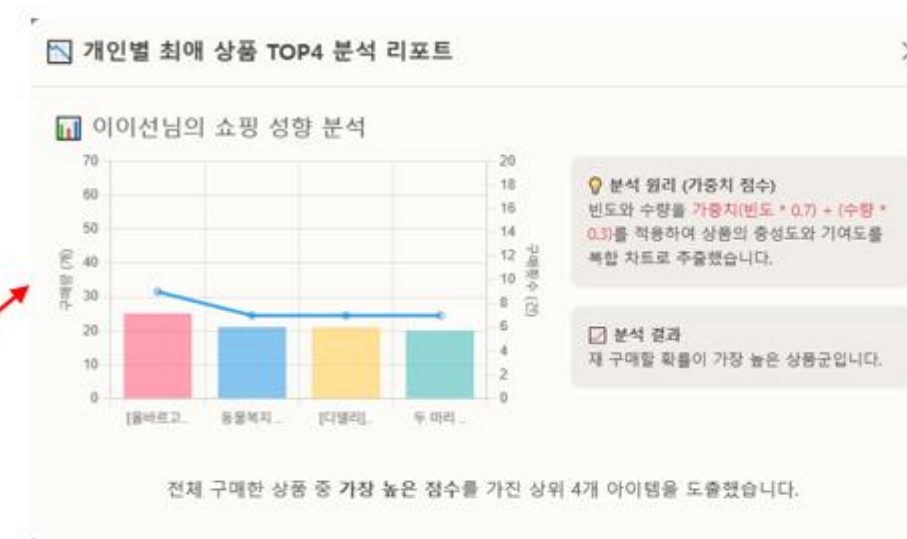


05-5 UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰) 프로토타입 설계화면(구매내역페이지)



주문/구매 화면

- 1 최근 및 기간 구매목록 조회
- 2 전체 구매이력으로 분석한 상품추천
- 3 상품추천 분석 리포트 믹스차트로 구현



06 UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰) 프로그램 구현



06-1

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

로그인/로그아웃

UNICOOK

저녁 뭐 먹지?

로그인 구매내역

⌚ 고단한 하루의 끝, 맛있는 저녁 한 끼 어때요?

UNICOOK 로그인

user_001

.....

로그인

시간대별 분석 리포트 확인하기

클릭하여 회원님께 딱 맞는 맞춤 상품 리포트를 확인해 보세요.

간편식 . 밀키트 . 샐러드

전채보기	샐러드샌드위치	떡볶이튀김순대
밀키트	도시락밥류	치킨·피자·핫도그·만두

```
78
79 #로그인 완료 (/loginok.do)
80 @app.route("/loginok.do", methods=["POST"])
81 def loginok() :
82     id = request.form["id"]
83     pw = request.form["pw"]
84     dao = UserDAO()
85     vo = dao.Login(id,pw)
86
87     if vo == None :
88         return "ERROR";
89     else :
90         session["login"] = { "id" : vo.id, "name" : vo.name }
91         return "OK";
```

```
8     def Login(self, id, pw) :
9
10         with DBManager() as db :
11             vo = None
12
13             sql = "select * from user "
14             sql += f"where id = '{id}' and pw = md5('{pw}')"
15
16             total = db.Select(sql)
17             if total >= 0 :
18                 vo = UserVO()
19                 vo.id = db.GetValue(0,"id")
20                 vo.name = db.GetValue(0,"name")
21             return vo
```

06-2

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

검색/카테고리

UNICOOK

저녁 뭐 먹지?



로그인

구매내역

🕒 고단한 하루의 끝, 맛있는 저녁 한 끼 어때요?



[마니커에프앤지] 바사삭 치킨 닭다리 420g
8,980원



[비비드키친] 저당 닭가슴살 브리또 6종 2개
입 (택1, NEW 옵션 추가)
7,800원



[애슬리] 오리자널 통살치킨
9,900원



[도우름] 이준셰프의 트러플 머쉬룸 수제 뇨끼
13,410원



시간대별 분석 리포트 확인하기

클릭하여 회원님께 딱 맞는 맞춤 상품 리포트를 확인해 보세요.

간편식 . 밀키트 . 샐러드

전체보기

샐러드·샌드위치

떡볶이·튀김·순대

밀키트

도시락·밥류

치킨·피자·핫도그·만두

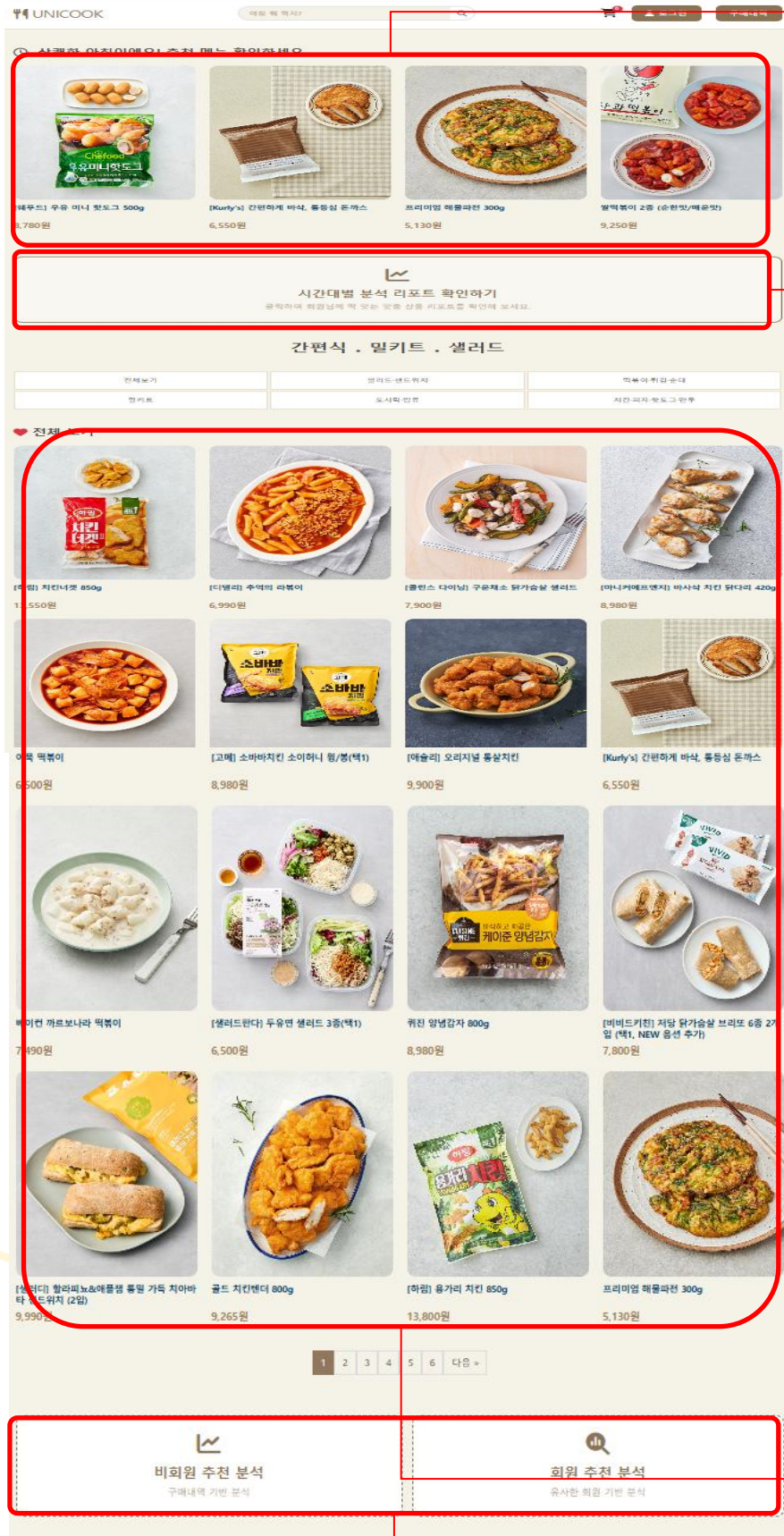
```
107 # 검색/카테고리 (/navi)
108 @app.route("/navi.do")
109 def navi() :
110     current_page = request.args.get('page', 1, type=int)
111     category = request.args.get("category", "0")
112     keyword = request.args.get('keyword')
113     dao = ItemDAO()
114     total, items = dao.GetList(current_page, category, keyword)
115
```

```
9 def GetList(self, page, category, keyword=None) :
10     """
11     게시판 목록
12     category:category_id ->
13     샐러드/샌드위치 : 1, 분식 : 2, 밀키트 : 3,
14     도시락/밥류 : 4, 치킨/피자/핫도그/만두 : 5
15     """
16     offset = (int(page) - 1) * 16
17     item = []
18     with DBManager() as db :
19         conditions = []
20         params = []
21         if category != "0" :
22             conditions.append("category_id = %s")
23             params.append(category)
24         if keyword :
25             conditions.append("item_name LIKE %s")
26             params.append(f"%{ keyword }%")
27
28         sql = "select count(code) as total "
29         sql += "from item "
30         if conditions :
31             sql += "where " + " AND ".join(conditions)
32
33         total = db.Select(sql, params)
34         total = db.GetValue(0, "total")
35
36         sql = "select * from item "
37         if conditions :
38             sql += "where " + " AND ".join(conditions)
39         sql += " order by hit desc "
40         sql += f"limit { offset }, 16 "
```


06-3

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

메인페이지(엔트리 포인트)



```

28 @app.route("/")
29 def main() :
30
31     login_info = session.get("login")
32     id = login_info.get("id") if login_info else None
33
34     dao = RecommendDAO()
35     time_data, slot, slot_range = dao.GetAiRecommend(id)
36
37     current_page = request.args.get('page', 1, type=int)
38     category = request.args.get("category", "0")
39
40     if id :
41         user_dao = UserDAO()
42         gender, age = user_dao.UserInfo(id)
43         dao.GetByCustom(id, gender, age)
44     else :
45         # 최근 1달 인기상품 분석
46         dao.GetByhit()
47
48     # 리스트 가져오기 실행
49     item_dao = ItemDAO()
50     total, items = item_dao.GetList(current_page, category)
51
52     # 페이지 6개씩 구현
53     total_pages = math.ceil(total / 16)
54     block_size = 5
55     start_page = ((current_page - 1) // block_size) * block_size + 1
56     end_page = start_page + block_size
57
58     if end_page > total_pages:
59         end_page = total_pages
60     return render_template("main.html", items = items,
61                           total_pages = total_pages,
62                           current_page = current_page,
63                           start_page = start_page,
64                           end_page = end_page,
65                           time_data = time_data,
66                           slot = slot,
67                           slot_range = slot_range
68     )

```

```

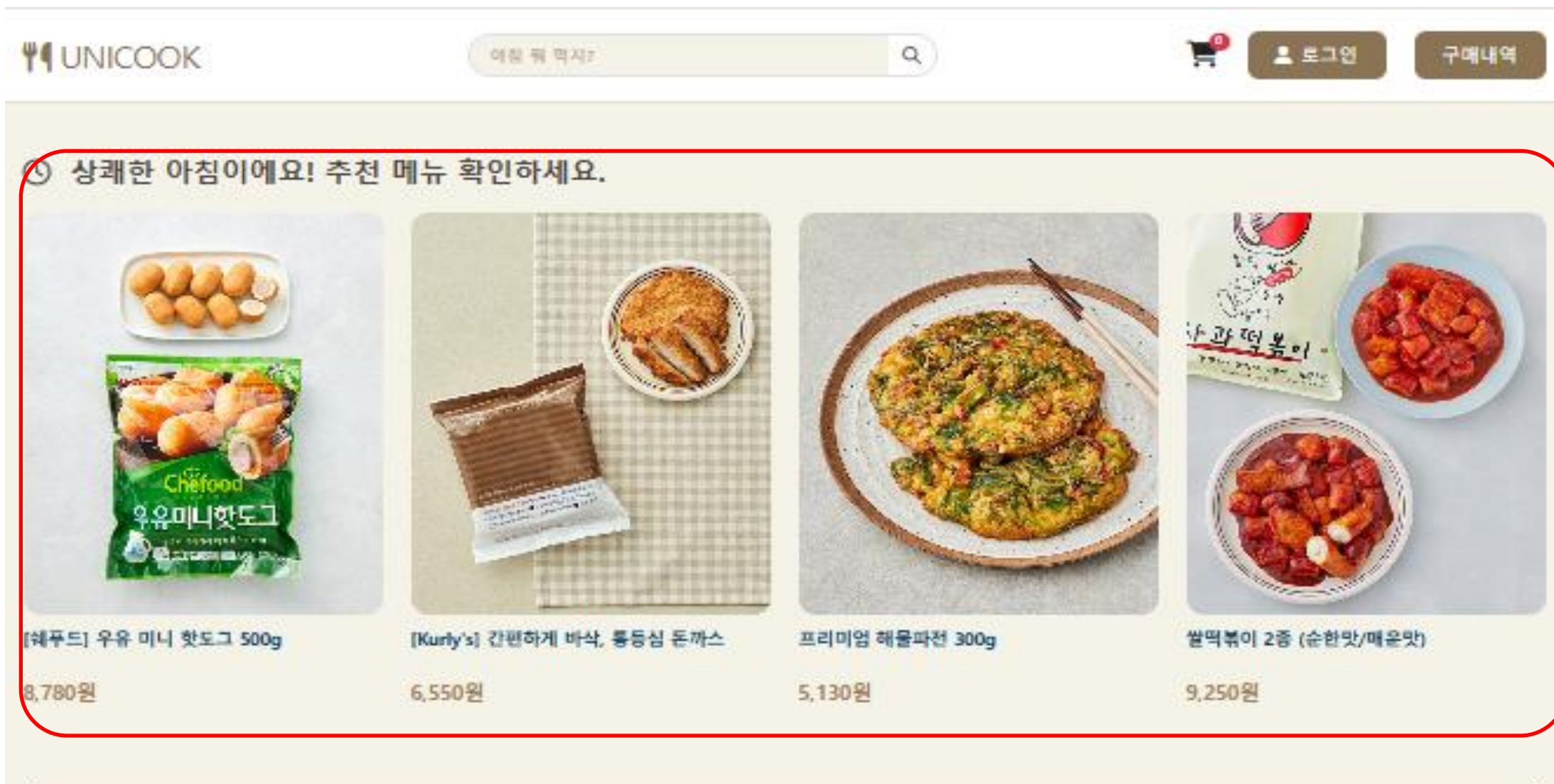
289 @app.route("/bunsuk.do")
290 def bunsuk(target = None) :
291
292     login_info = session.get("login")
293     id = login_info.get("id") if login_info else None
294     check_id = True if id else False
295
296     if not target :
297         target = request.args.get("target", "main")
298
299     if target == "main" :
300         time_dao = RecommendDAO()
301         time_data, slot, slot_range = time_dao.GetAiRecommend(id)
302         return render_template("bunsuk_main.html",
303                               target = target,
304                               check_id = check_id,
305                               time_data = time_data,
306                               slot = slot,
307                               slot_range = slot_range
308                               )
309
310     if target == "mainsubN" :
311         return render_template("/bunsuk_mainsubN.html")
312
313     if target == "mainsubY" :
314         return render_template("/bunsuk_mainsubY.html")
315

```


06-3

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

메인페이지(시간대 상품 추천 분석)



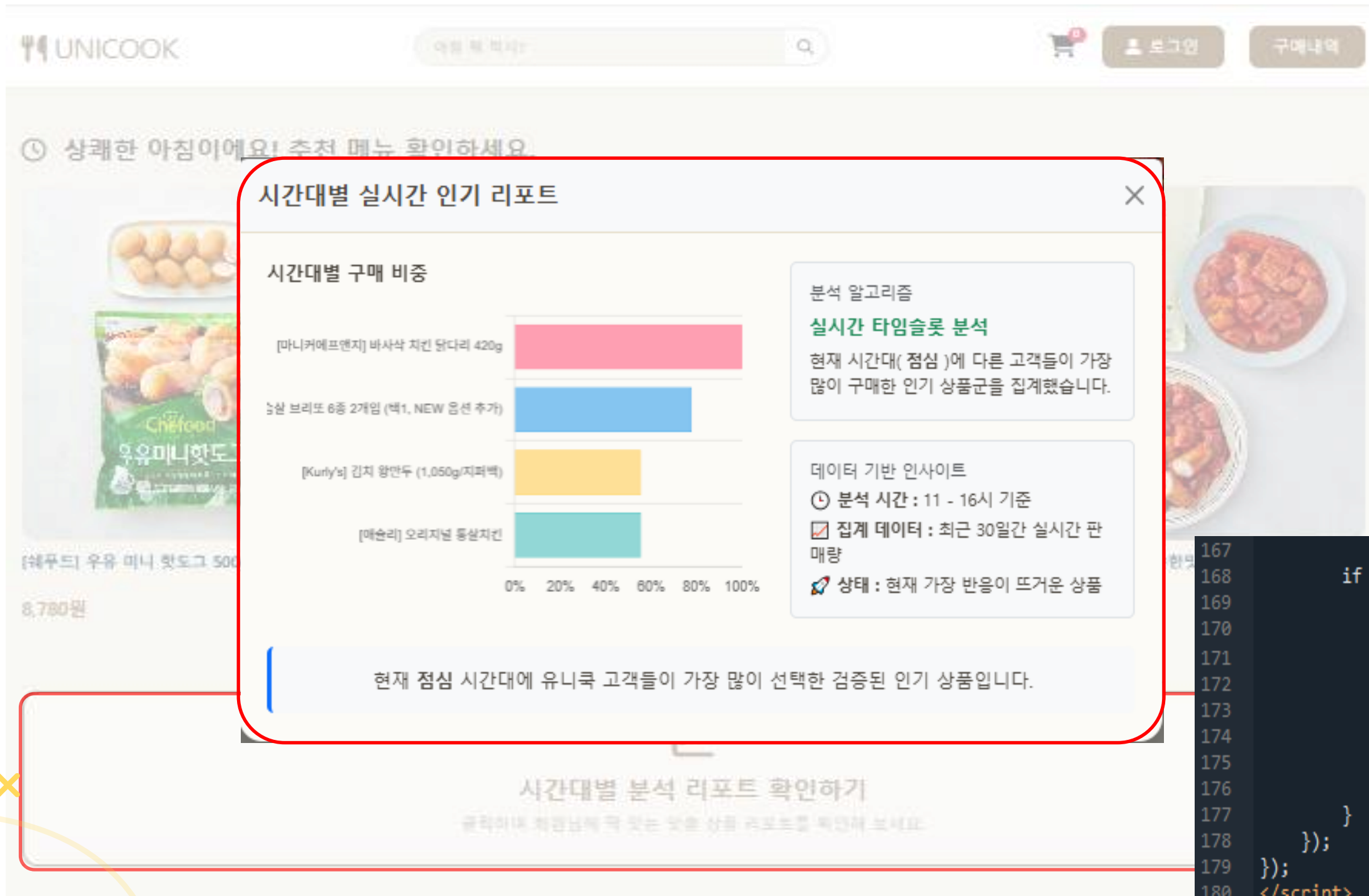
시간대별 분석
클릭하여 회원님께 딱 맞는

```
682
683
684 now_hour = datetime.now().hour
685 if 6 <= now_hour < 11 :
686     slot, t_range, slot_range = "morning", (6, 10), "6 - 10"
687 elif 11 <= now_hour < 17 :
688     slot, t_range, slot_range = "lunch", (11, 16), "11 - 16"
689 elif 17 <= now_hour < 21 :
690     slot, t_range, slot_range = "dinner", (17, 20), "17 - 20"
691 else:
692     slot, t_range, slot_range = "night", (21, 5), "21 - 5"
```


06-3

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

메인페이지(시간대 상품 추천 분석 시각화)



```

104 function ShowBunsuk(target)
105 {
106     $.ajax({
107         url : "/bunsuk.do?target=" + target,
108         type : "get",
109         data : {target : target},
110         async : true,
111         success : function(result)
112         {
113             $("#popupModal").html(result);
114             var obj = document.getElementById("analysisModal");
115             var modal = new bootstrap.Modal(obj);
116             modal.show();
117         },
118         error : function(request, status, error)
119         {
120         }
121     });
122 }
123

```

```

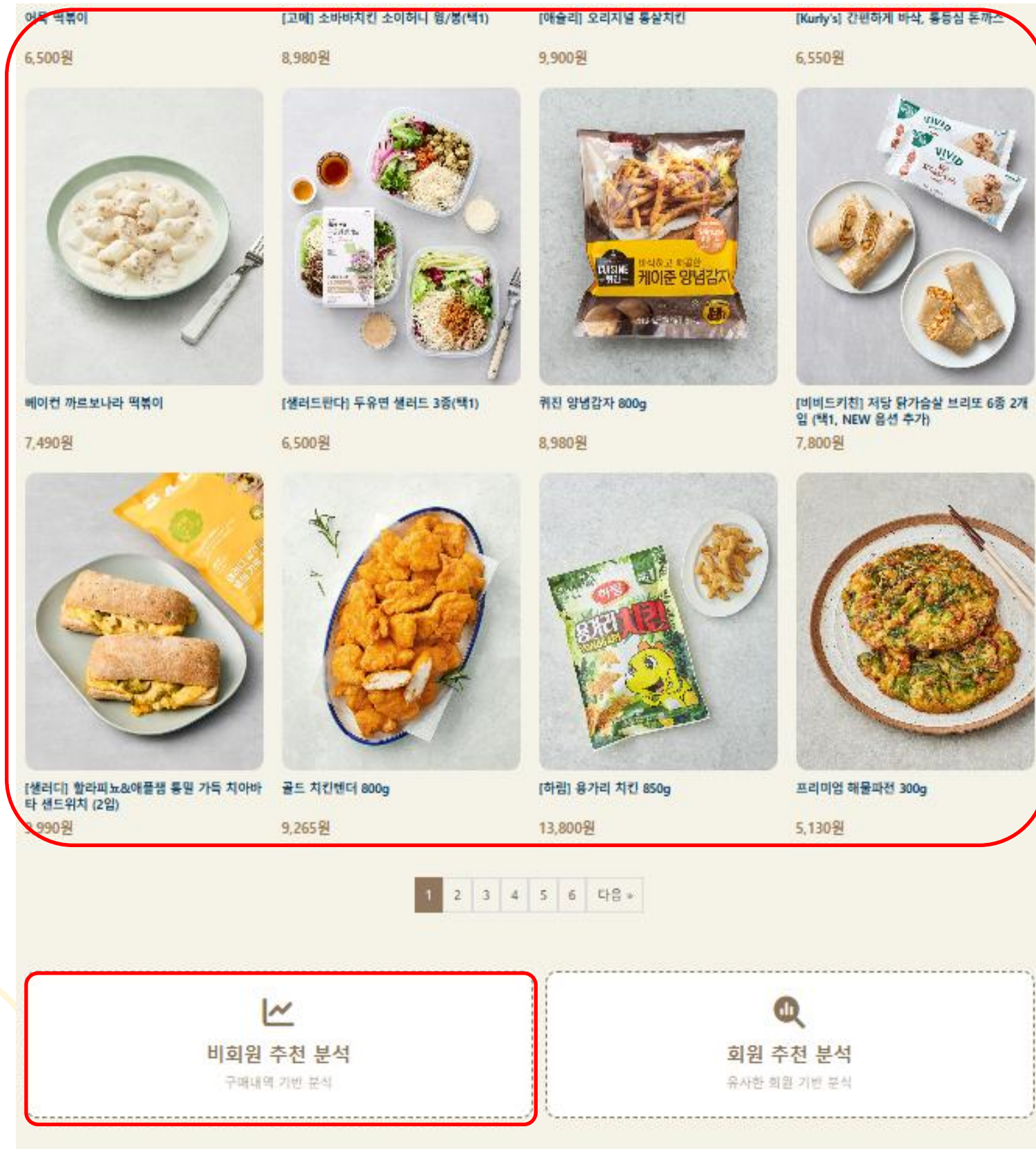
167
168
169
170 // const avg = (scores.reduce((a, b) => a + b, 0) / scores.length) * 100; 아래 함수와 같음
171 var sum = scores.reduce(function(a, b) {
172     return a + b;
173 }, 0);
174 var avg = (sum / scores.length) * 100;
175
176 $('#avg-score-display').text(avg.toFixed(1));
177
178 });
179 });
180 </script>

```

06-3

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

메인페이지(비회원 상품 추천 분석)



```
df_concat['code'] = df_concat['code'].astype(str)
```

```
# 가중치 점수 계산 (최근 인기템 지수)
```

```
# 최근 한달 구매수에 더 높은 가중치(0.7)를 두고 구매 전환비(전체 구매수 / 전체 조회수 * 0.3)를 더하여 최근 인기상품을 분석함.
```

```
df_concat['hit'] = (df_concat['total_1m'] * 0.7) + (df_concat['total_all'] / df_concat['view'] * 0.3)
```

```
# 인기순(hit) 내림차순 정렬
```

```
df_concat = df_concat.sort_values(by='hit', ascending=False)
```

```
# item 테이블 삽입위한 hit 값 소수점 4자리로 정리
```

```
df_concat['hit'] = df_concat['hit'].apply(lambda x: round(float(x), 4))
```

```
# 결측값 있는 행 모두 제거
```

```
df_concat = df_concat.dropna()
```

```
# item 테이블 hit 컬럼 수정
```

```
for i in range(len(df_concat)) :
    code = df_concat.iloc[i]["code"]
    hit = df_concat.iloc[i]["hit"]
    sql = "update item "
    sql += f"set hit = {hit} "
    sql += f"where code = {code} "
    db.RunSQL(sql)
```

```
df_chart = df_concat.copy()
```

```
df_chart = df_chart.drop(columns=['code', 'total_all', 'view'])
```

```
df_chart['CVR'] = df_concat['total_all'] / df_concat['view']
```

```
df_chart['CVR'] = df_chart['CVR'].apply(lambda x: round(float(x), 4))
```

```
return df_chart
```


06-3

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

메인페이지(비회원 상품 추천 분석 시각화)



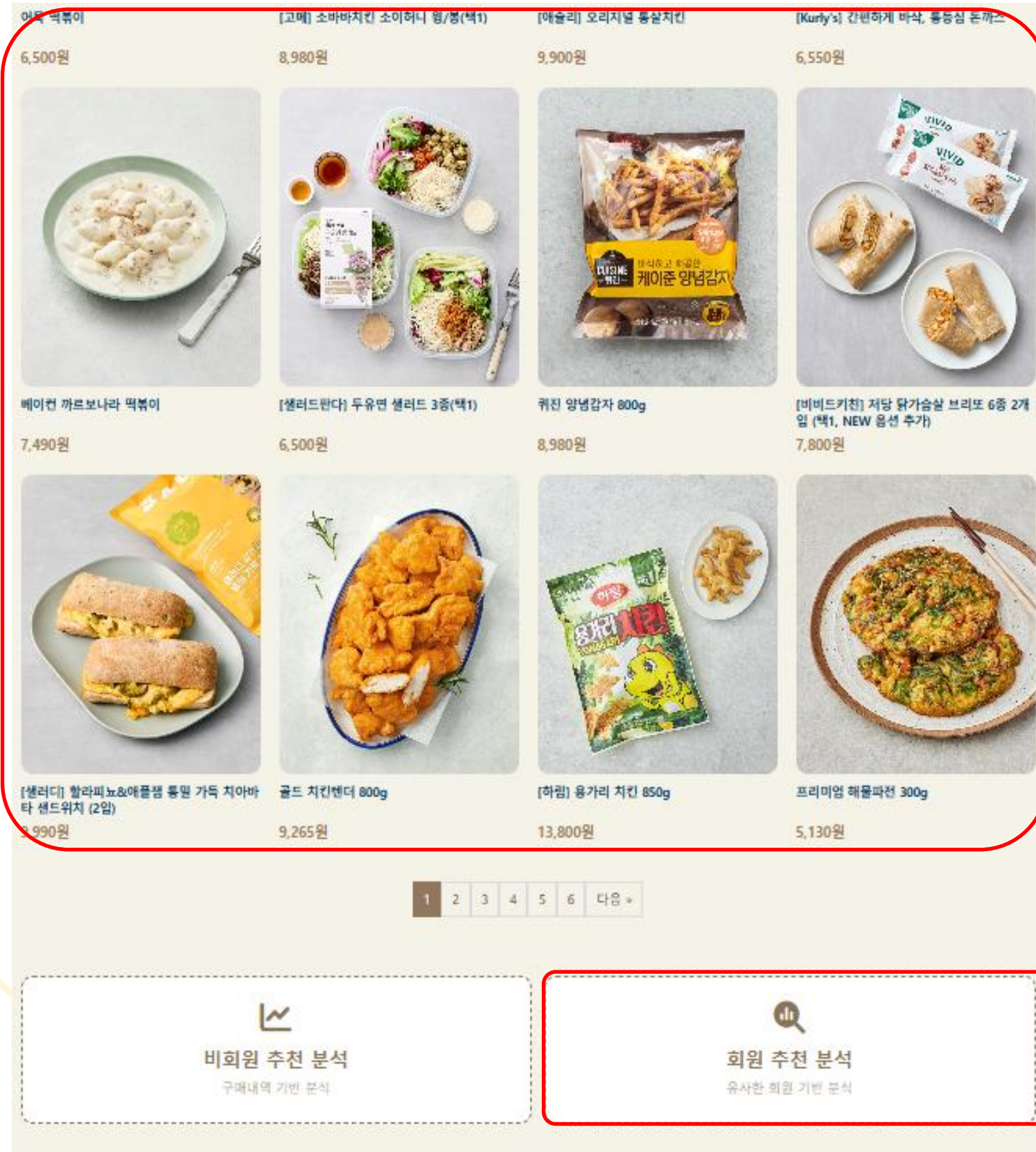
```

419 # 메인하단 분석 시각화(비회원)
420 @app.route("/bubble.do")
421 def bubble():
422     dao = RecommendDAO()
423     df_chart = dao.GetByhit()
424
425     # 가중치 점수 상위 6개만 추출
426     top = df_chart.sort_values('hit', ascending=False).head(6)
427
428     # 데이터프레임 -> 딕셔너리 변환
429     chart_data = top.to_dict(orient='records')
430
431     return jsonify(chart_data)
    
```

06-3

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

메인페이지(회원 상품 추천 분석)



```
# 2. 성별과 나이대를 원-핫 인코딩(One-Hot Encoding)으로 변환
# 나이와 성별도 '유사도' 계산에 참여하게 만들기 위함입니다.
user_features = df_filt_data[['id', 'gender', 'age_group']].drop_duplicates().set_index('id')
user_features_dummy = pd.get_dummies(user_features, columns=['gender', 'age_group'])

# 3. 상품 구매 이력 피벗 테이블 생성
item_matrix = df_filt_data.pivot_table(index='id', columns='item_name', values='cnt', fill_value=0)

# 4. 두 데이터 결합 (구매 이력 + 유저 정보)
# 이렇게 하면 '구매 패턴'과 '나이/성별'이 모두 포함된 유저 벡터가 만들어집니다.
final_matrix = pd.concat([item_matrix, user_features_dummy], axis=1).fillna(0)
print(final_matrix)
# 유저 간 코사인 유사도 계산
user_sim = cosine_similarity(final_matrix)
user_sim_df = pd.DataFrame(user_sim, index=final_matrix.index, columns=final_matrix.index)

# 로그인한 유저와 유사한 상위 N명 추출
sim_scores = user_sim_df[target_id].sort_values(ascending=False)
sim_users = sim_scores.iloc[1:11].index # 0번은 자기자신이므로 1번부터 10번까지

# 유사한 유저 10명의 구매 데이터만 필터링
sim_user_interactions = item_matrix.loc[sim_users]

# 유사도 점수를 가중치로 활용하여 상품별 점수 계산
rec_scores = pd.Series(dtype=float)

for user in sim_users:
```


06-3

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

메인페이지(회원 상품 추천 분석 시각화)

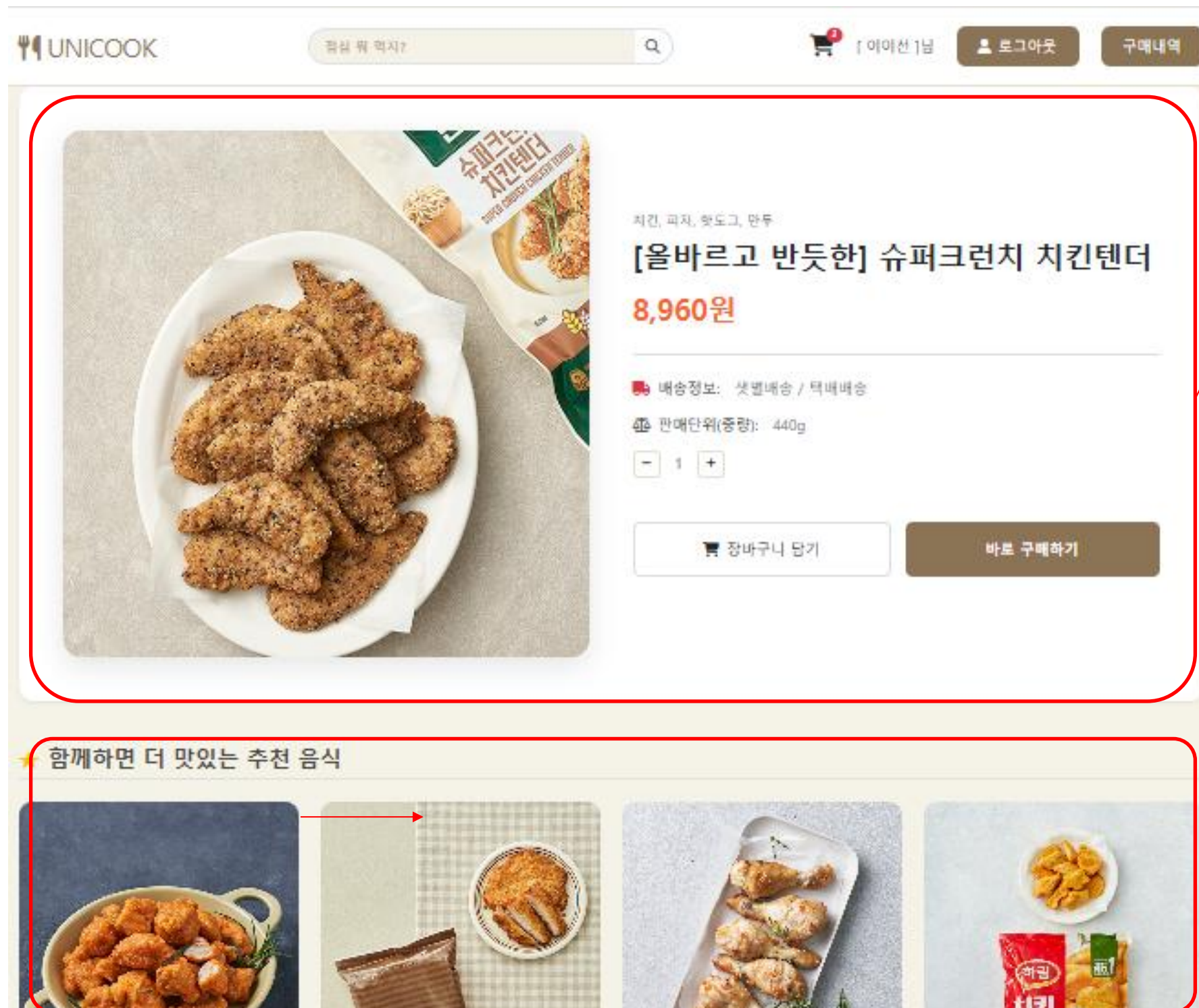


```
433 # 메인하단 분석 시각화(회원)
434 @app.route("/heatmap.do")
435 def heatmap():
436     id = session["login"]["id"]
437
438     user_dao = UserDao()
439     gender, age = user_dao.UserInfo(id)
440
441     dao = RecommendDAO()
442     sim_buy = dao.GetByCustom(id, gender, age)
443
444     # 데이터프레임 -> 딕셔너리 변환
445     chart_data = sim_buy.reset_index().to_dict(orient='records')
446
447     return jsonify(chart_data)
```


06-4

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

상세페이지(엔트리 포인트)



```

125 # 상세페이지
126 @app.route("/view.do")
127 def view():
128     code = request.args.get("code", "")
129     if code == "":
130         return redirect("/")
131     dao = ItemDAO()
132     item = dao.View(code)
133     return render_template("view.html", item=item)
134

```

```

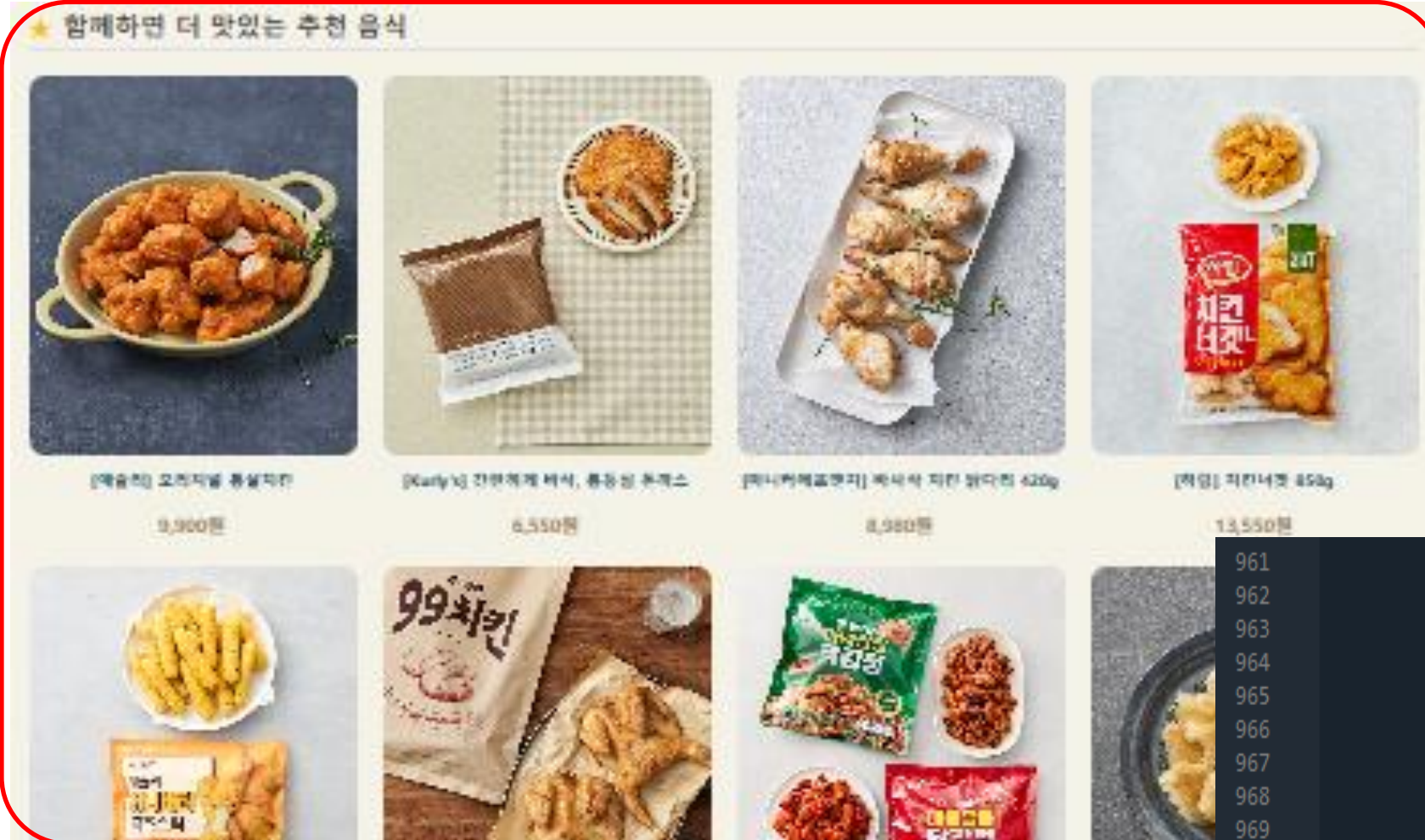
289 @app.route("/bunsuk.do")
290 def bunsuk(target = None):
291
292     login_info = session.get("login")
293     id = login_info.get("id") if login_info else None
294     check_id = True if id else False
295
296     if not target:
297         target = request.args.get("target", "main")
298
299     if target == "view":
300         view_dao = RecommendDAO()
301         view_dao.MakePersonalBestRecommendations(id, algo_type = "view")
302         total, items = buy_dao.GetByUserFrequency(id, n=4, algo_type = "view")
303         if int(total) > 0:
304             return render_template("/bunsuk_view.html",
305                                   target = target,
306                                   total = total,
307                                   items = items
308             )
309

```


06-4

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

상세페이지(연관분석 상품 추천 분석)



```

947
948     # 사용자 - 상품 매트릭스 생성
949     user_item_matrix = df.pivot_table(index = "id", columns = "code", values = "qty", fill_value = 0)
950     # 아이템 - 사용자 매트릭스로 뒤집기
951     item_user_matrix = user_item_matrix.T
952
953     # 상품 간 코사인 유사도 계산
954     item_sim = cosine_similarity(item_user_matrix)
955     item_sim_df = pd.DataFrame(item_sim, index = item_user_matrix.index, columns = item_user_matrix.index)
956
957     sim_items = item_sim_df[current_code].sort_values(ascending = False)[1:top_k+1]
958
959     predictions = []
  
```

```

961     for sim_code, score in sim_items.items() :
962
963         predictions.append({
964             "id"      : target_id,
965             "code"     : sim_code,
966             "score"    : round(float(score), 4),
967             "algo_type" : algo_type
968         })
969     ndf = pd.DataFrame(predictions)
970
971     # 기존 추천정보 삭제
972     sql = "delete from score "
973     sql += f"where id = '{target_id}' and algo_type = '{algo_type}' "
974     db.RunSQL(sql)
975
976     for i in range(0, len(ndf)) :
977         code = ndf.iloc[i]["code"]
978         score = ndf.iloc[i]["score"]
979         sql = "insert into score "
980         sql += "(id, code, score, algo_type) "
981         sql += "values "
982         sql += f"('{target_id}', {code}, {score}, '{algo_type})' "
983         db.RunSQL(sql)
984
  
```


06-4

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

상세페이지(연관분석 상품 추천)



[사세] 버팔로 왕 골드 1kg

18,900원



두 마리 99치킨 (에어프라이어용)

9,900원



[백제원] 국내산 닭과 전복을 사용한 영양 해신탕

14,235원



이탈리안 루꼴라 샐러드믹스 2종

4,990원



[비비고] 왕교자 1.4KG

14,980원



[Chai797] 딤섬 새우 창편 300g

8,900원



[하림] 용가리 치킨 850g

13,800원



나의 식단 분석 리포트 확인하기

클릭하여 회원님께 딱 맞는 맞춤 상품 리포트를 확인해 보세요.

```

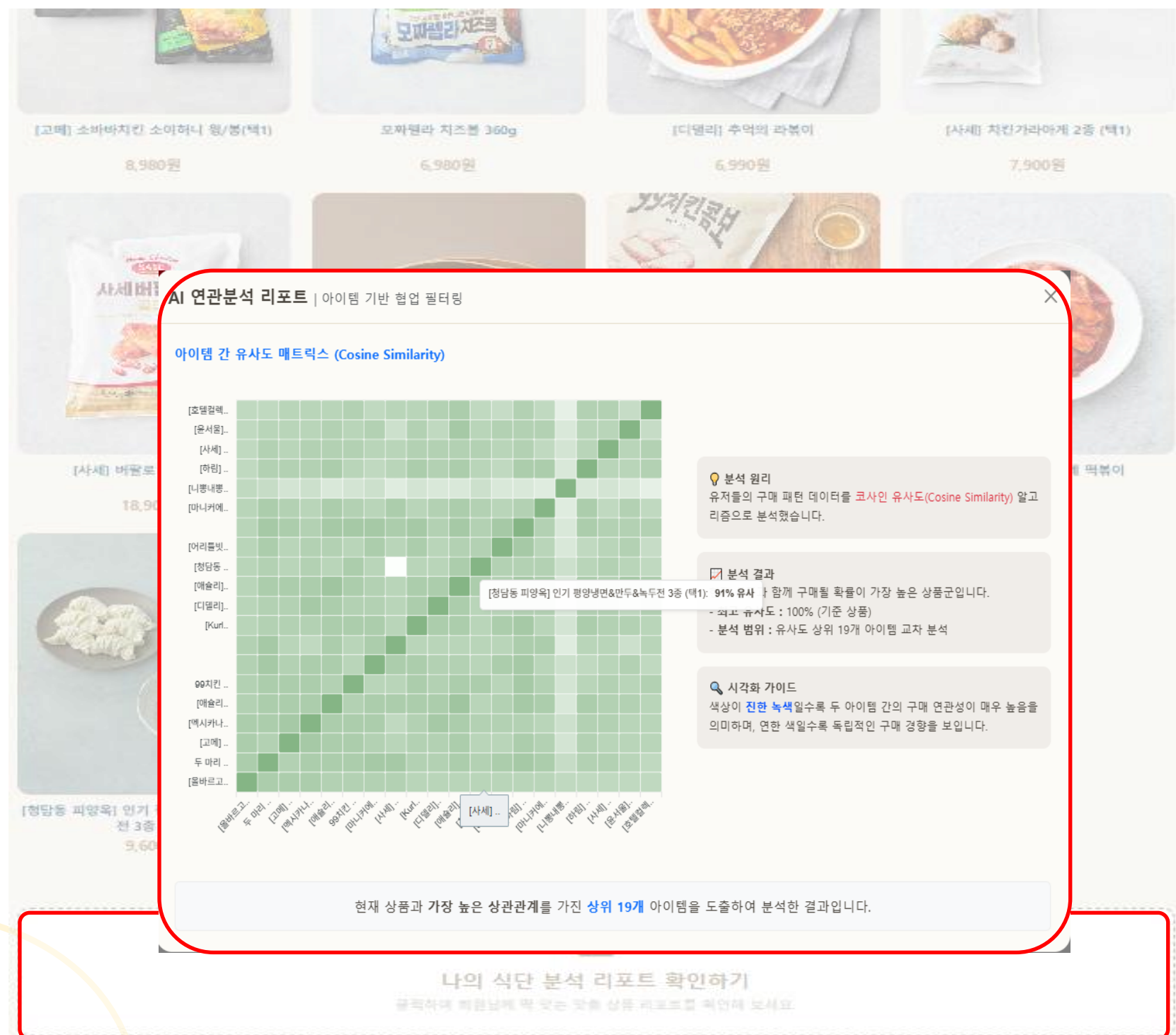
157 $.ajax({
158   url : "/recommend.do",
159   type : "get",
160   dataType: "json",
161   data : sendData,
162   async : true,
163   success : function(response)
164   {
165     let html = "";
166
167     // main, main_sub, cart 에서는 배열 반환
168     // view 에서는 객체 반환
169     let items = response.items ? response.items : response;
170
171     if (response.chart_data) {
172       $("#chartData").val(JSON.stringify(response.chart_data));
173     }
174
175     items.forEach(function(item)
176     {
177       html += `
178         <div class="col-6 col-md-3">
179           <a href="/view.do?code=${item.code}">
180             <div class="product-card">
181               
182               <div class="product-name">${item.item_name}</div>
183               <div class="product-price">${item.price}</div>
184             </div>
185           </a>
186         </div>
187       `;
188     });
189     $('#reco_items').html(html);
190     if(target == "purchase"){
191       window.scrollTo(0, document.body.scrollHeight);
192     }
193   },
194   error : function(request, status, error)
195   {
196     alert("실패");
197   }
198 });
199

```

06-4

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

상세페이지(연관분석 상품 추천 시각화)



```

104 function ShowBunsuk(target)
105 {
106
107     let sendData = { target: target };
108
109     //'view' 호출 시 현재 상품 코드를 추가
110     if (target == "view") {
111         const code = $("#code").val();
112         if (code) {
113             sendData.code = code;
114         }
115     }
116
117     $.ajax({
118         url : "/bunsuk.do",
119         type : "get",
120         data : sendData,
121         async : true,
122         success : function(result)
123         {
124             $("#popupModal").html(result);
125             var obj = document.getElementById("analysisModal");
126             var modal = new bootstrap.Modal(obj);
127             modal.show();
128             if(target == "view"){
129                 const chartData = $("#chartData").val();
130                 if(chartData){
131                     const chartObj = JSON.parse(chartData);
132                     ShowChart(chartObj);
133                 }
134             }
135
136         },
137         error : function(request, status, error)
138         {
139         }
140     });
141 }
  
```


UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰) 장바구니(엔트리 포인트)



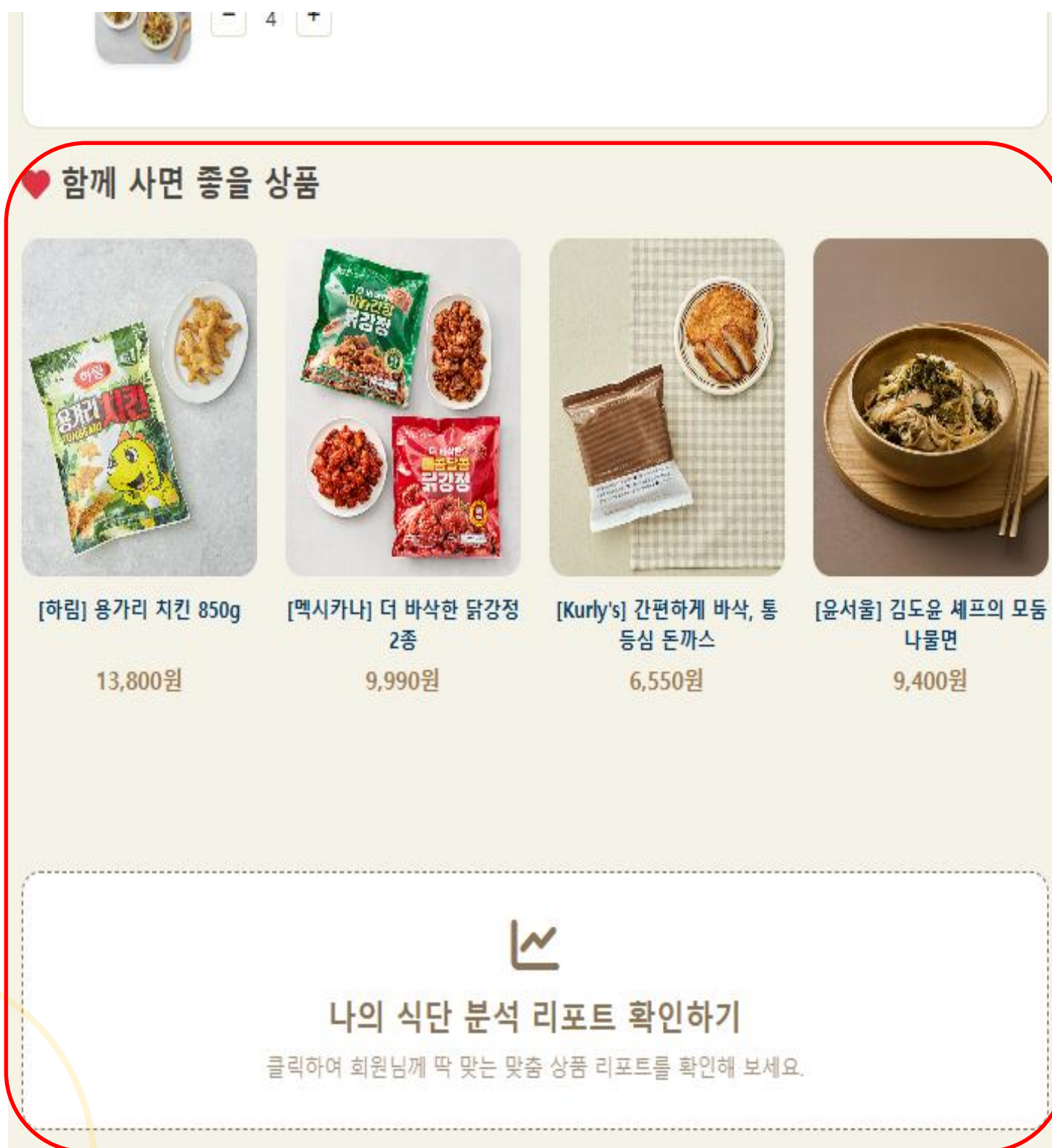
```

308 @app.route("/bunsuk.do")
309 def bunsuk(target = None) :
310
311     login_info = session.get("login")
312     id = login_info.get("id") if login_info else None
313     check_id = True if id else False
314
315     if not target :
316         target = request.args.get("target","main")
317
318     if target == "cart" :
319         cart_dao = RecommendDAO()
320         cart_dao.CartAiRecommend(id, algo_type = "cart")
321         total, items = cart_dao.GetByUserFrequency(id, n=4, algo_type = "cart")
322         if int(total) > 0 :
323             return render_template("bunsuk_cart.html",
324                                     target = target,
325                                     total = total,
326                                     items = items
327                                 )

```

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

장바구니(연관분석 상품 추천)



총 결제금액

```

872
873
874     # 아이템 간 유사도 계산 (상품 간 코사인 유사도)
875     item_sim = cosine_similarity(user_item_matrix.T)
876     item_sim_df = pd.DataFrame(item_sim, index=user_item_matrix.columns, columns=user_item_matrix.columns)
877
878     # 로그인 유저의 현재 장바구니 아이템 가져오기
879     current_cart_items = df_cart[df_cart["id"] == target_id]["code"].tolist()
880
881     if not current_cart_items :
882         return "장바구니가 비어 있어 추천 불가"
883
884     # 장바구니 아이템과 유사한 상품 점수 합계 구하기
885     # 장바구니 담긴 상품들이 다른 상품들과 갖는 유사도를 모두 더함
886     recom_scores = item_sim_df[current_cart_items].sum(axis = 1)
887
888     # 결과값 데이터프레임 생성
889     recom_df = recom_scores.reset_index()
890     recom_df.columns = ["code", "score"]
891
892     # 필터링 로직
893     # 이미 장바구니에 담긴 상품 제외
894
895     exclude_items = set(current_cart_items)
896

```

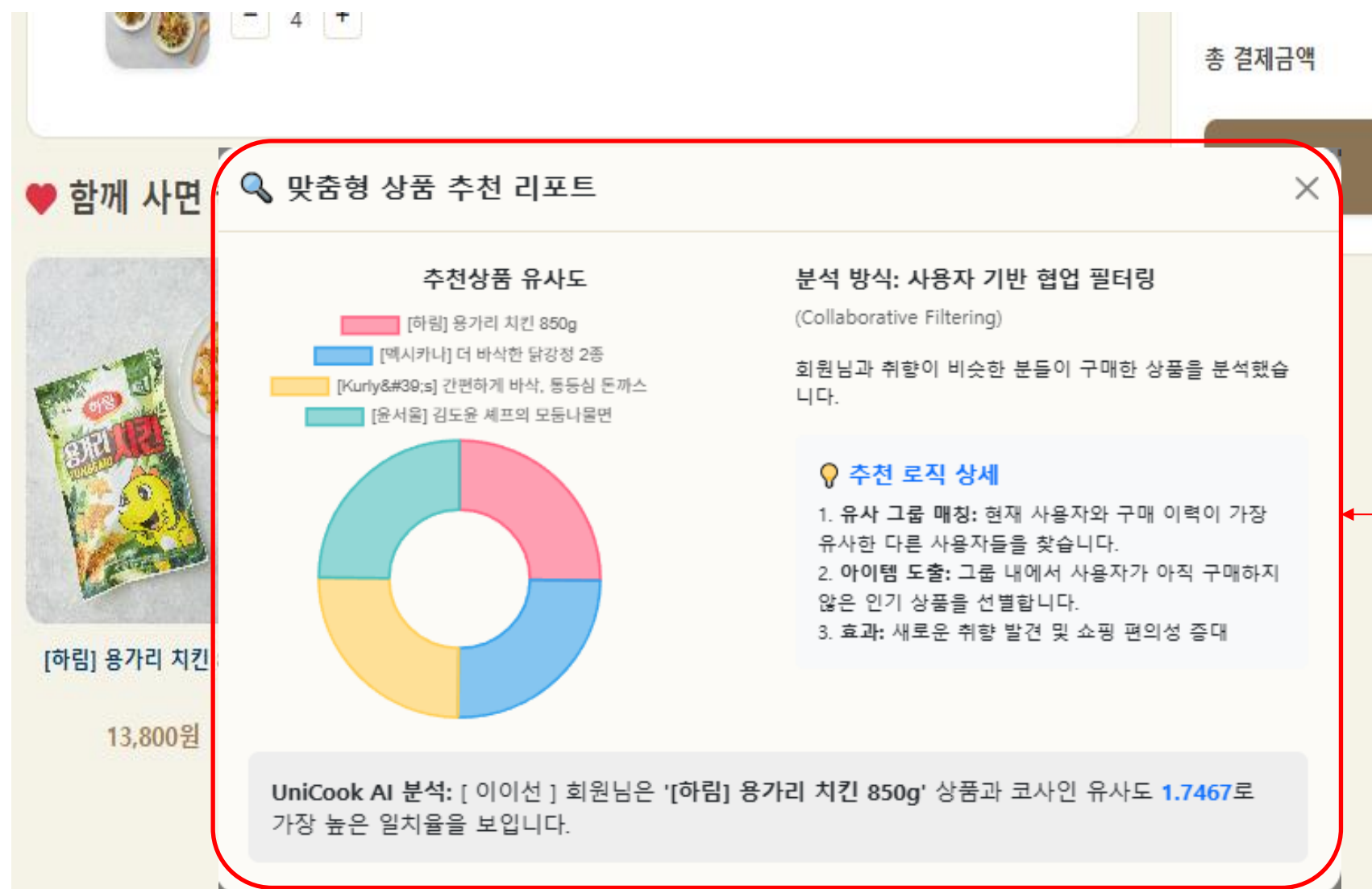
```

858
859     # 기존 추천정보 삭제
860     sql = "delete from score "
861     sql += f"where id = '{target_id}' and algo_type = '{algo_type}'"
862     db.RunSQL(sql)
863
864     # 데이터 전처리
865     df_buys = df_buys.copy()
866     df_buys["code"] = df_buys["code"].astype(str)
867     df_cart = df_cart.copy()
868     df_cart["code"] = df_cart["code"].astype(str)
869

```


UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

장바구니(연관분석 상품 추천 시각화)



나의 식단 분석 리포트 확인하기
클릭하여 회원님께 딱 맞는 맞춤 상품 리포트를 확인해 보세요.

```

104 function ShowBunsuk(target)
105 {
106
107     let sendData = { target: target };
108
109     //'view' 호출 시 현재 상품 코드를 추가
110     if (target == "view") {
111         const code = $("#code").val();
112         if (code) {
113             sendData.code = code;
114         }
115     }
116
117     $.ajax({
118         url : "/bunsuk.do",
119         type : "get",
120         data : sendData,
121         async : true,
122         success : function(result)
123         {
124             $("#popupModal").html(result);
125             var obj = document.getElementById("analysisModal");
126             var modal = new bootstrap.Modal(obj);
127             modal.show();
128             if(target == "view"){
129                 const chartData = $("#chartData").val();
130                 if(chartData){
131                     const chartObj = JSON.parse(chartData);
132                     ShowChart(chartObj);
133                 }
134             }
135
136         },
137         error : function(request, status, error)
138         {
139         }
140     });
141 }

```

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰) 구매내역(엔트리 포인트)



```

564 def RecommendItem(self, userid, algo_type) :
565     item = []
566     with DBManager() as db :
567         conditions = []
568         params      = []
569         conditions.append("algo_type = %s")
570         params.append(algo_type)
571         if userid :
572             conditions.append("id = %s")
573             params.append(userid)
574         sql = "select i.code, i.image, i.item_name, i.price "
575         sql += "from score s "
576         sql += "join item i "
577         sql += "on s.code = i.code "
578         if conditions :
579             sql += "where " + " AND ".join(conditions)
580         sql += " order by score desc "
581
582         count = db.Select(sql, params)
583         for n in range(count) :
584             vo = RecommendVO()
585             vo.code      = db.GetValue(n, "code")
586             vo.image     = db.GetValue(n, "image")
587             vo.item_name = db.GetValue(n, "item_name")
588             vo.price     = f"{db.GetValue(n, 'price'):,}€"
589             item.append(vo)
590     return item

```


06-6

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

구매내역(가중치 상품 추천 분석 데이터)

[셀러드판다] 부라타 햄 치아바타 샌드위치
단가 8,200원 | 총 결제금액 8,200원

2026-04-08 15:49:46 주문번호: 100010

[콜린스 이자카야] 오뎅나베
단가 11,900원 | 총 결제금액 23,800원

1 2 3 4 5 다음 >

★ AI 추천상품

[올바르고 반듯한] 슈퍼크런치 치킨텐더
8,960원

동물복지 치킨 너겟 오리지날
7,980원

[디캘리] 추억의 라볶이
6,990원

두 마리 99치킨 (에어프라이어용)
9,900원

구매 내역을 기반으로 한 TOP4 상품 확인하기

클릭하면 상단에 추천 상품이 나옵니다.

```

520 # 2. 상품별 구매 빈도와 누적 수량 계산
521 # frequency: 서로 다른 주문번호의 개수 (얼마나 자주 찾았나)
522 # total_qty: 수량의 합계 (얼마나 많이 샀나)
523 best_stats = user_orders.groupby('code').agg(
524     frequency=('bno', 'nunique'),
525     total_qty=('qty', 'sum')
526 ).reset_index()
527
528 # 3. 가중치 점수 계산 (인생템 지수)
529 # 빈도에 더 높은 가중치(0.7)를 두어 반복적으로 구매한 습관을 중시함
530 best_stats['raw_score'] = (best_stats['frequency'] * 0.7) + (best_stats['total_qty'] * 0.3)
531
532 # 4. score 테이블 저장을 위해 0~1 사이로 정규화 (Min-Max Scaling)
533 # 추천 점수는 상대적으로 가장 점수가 높은 상품을 1점으로 만들
534 max_score = best_stats['raw_score'].max()
535 min_score = best_stats['raw_score'].min()
536
537 if max_score == min_score:
538     best_stats['score'] = 1.0 # 모든 상품 점수가 같으면 1점으로 통일
539 else:
540     best_stats['score'] = (best_stats['raw_score'] - min_score) / (max_score - min_score)
541
542 # 5. score 테이블 형식으로 가공
543 best_stats['id'] = target_user_id
544 best_stats['algo_type'] = algo_type
545 best_stats['score'] = best_stats['score'].apply(lambda x: round(float(x), 4))
546
547 # 점수 높은 순으로 정렬하여 상위 4개 반환
548 result = best_stats[['id', 'code', 'score', 'algo_type']]
549 ndf = result.sort_values(by='score', ascending=False).head(4)
550

```

06-6

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

구매내역(가중치 상품 추천)



```

166
167 // main, main_sub, cart 에서는 배열 반환
168 // view 에서는 객체 반환
169 let items = response.items ? response.items : response;
170
171 if (response.chart_data) {
172     $("#chartData").val(JSON.stringify(response.chart_data));
173 }
174
175 items.forEach(function(item)
176 {
177     html += `
178         <div class="col-6 col-md-3">
179             <a href="/view.do?code=${item.code}">
180                 <div class="product-card">
181                     
182                     <div class="product-name">${item.item_name}</div>
183                     <div class="product-price">${item.price}</div>
184                 </div>
185             </a>
186         </div>
187     `;
188 });
189 $('#reco_items').html(html);
190 if(target == "purchase"){
191     window.scrollTo(0, document.body.scrollHeight);
192 }
193 },
194 error : function(request, status, error)
195 {
196     alert("실패");
197 }
198 });
199 }
    
```

06-6

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

구매내역(가중치 상품 추천 시각화)



```

437 # 구매내역 분석 시각화
438 @app.route("/mixed.do")
439 def mixed():
440     id = session["login"]["id"]
441
442     dao = RecommendDAO()
443     items = dao.GetChartmixed(id)
444     dict_list = [
445         {
446             "item_name": vo.item_name,
447             "freq": vo.count,
448             "qty": vo.qty
449         } for vo in items
450     ]
451     return jsonify(dict_list)

```

07 UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰) 자체 평가 의견



07

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

자체 평가 의견

프로젝트 결과물에 대한 완성도 평가 -사전기획관점에서 볼 때 (8점/10점 만점)-

- 초기에 기획한 분석 도구를 모두 사용하지 못하였으나, 70% 이상 구현하였음.
- 초기에 기획한 시각화 도구를 모두 사용하였음.
- context manager protocol을 사용하는 with 구문을 사용함으로써 메모리 누수를 방지함.

추후 개선점이나 보완할 점 - 프로젝트 결과물 -

- entry point의 코드를 Blueprint를 사용하여 체계화 하여 가독성과 유지보수성을 증가시킬 필요가 있음.
- 데이터 분석에 대한 도메인 지식을 깊이 하여 다양한 분석 도구를 사용해 보는 것이 필요함.

잘한 점과 아쉬운 점 -개인 및 팀 관점에서 볼 때-

〈잘한 점〉

- 팀협업이 잘 됨(매일 회의 후 회의록 작성, 코드완성 후 코드리뷰함)
- Python기반의 데이터 흐름을 정확히 이해하고 구조적으로 설계하여 코드의 유지보수성과 확장성에 대한 이해도 향상되었음.
- 다양한 데이터 분석에 대해 서로 공부한 것을 공유하여 도메인 지식을 쌓는데 서로에게 도움을 주었고 프로젝트를 진행하면서 보다 더 적절한 데이터 분석 도구를 사용할 수 있었음.

〈아쉬운 점〉

- 의견 조율은 매우 잘됐으나 이해하는 과정에서 다시 확인하지 않아 약간의 코드 중복으로 인해 수정하는 과정을 겪었음.

느낀 점 및 경험한 성과 - 프로젝트 수행시 -

〈느낀 점〉

- 이전 프로젝트와 비교하였을 때, 문법이나 함수를 적용하는 것이 더 쉬웠으나 결과물을 도출해낼 때는 조금 더 깊은 지식이 필요함을 느낌.
- 오류를 해결하는 과정에서 정답을 찾아가는 디버깅 역량이 크게 향상됨.

〈경력 계획과의 연관성〉

- 두 번의 프로젝트를 진행하면서 다른 언어를 배우는 것에 대한 자신감이 생김.
- 향후 실무에서 데이터 분석을 해야 하는 상황에서 보다 빠르게 적응할 수 있음.

UNICOOK(개인 구매패턴을 활용한 지능형 상품 추천 쇼핑몰)

이상으로 A_TEAM의 최종발표를 마칩니다.

